

深度学习入门

WPC

2026 年 6 月 18 日

目录

第一部分 深度学习	7
第一章 感知机	9
1.0.1 感知机简介	9
1.0.2 实现与门	9
1.0.3 局限性	10
第二章 神经网络	13
2.0.1 神经网络简介	13
2.0.2 损失函数	14
第三章 计算图	15
3.0.1 计算图简介	15
3.0.2 反向传播	16
第四章 CNN	19
4.0.1 CNN 简介	19
4.0.2 卷积层	19
4.0.3 池化层	21
第五章 自然语言处理	23
5.0.1 单词表示	23
5.0.2 基于同义词词典的方法	23
5.0.3 基于统计的方法	23
5.0.4 基于推理的方法	24
第六章 分词	25
6.0.1 分词简介	25
6.0.2 BPE	25
6.0.3 BBPE	26
6.0.4 WordPiece	26

第七章 RNN	27
7.0.1 语言模型	27
7.0.2 RNN 介绍	27
7.0.3 语言模型的评估	29
7.0.4 梯度消失和梯度爆炸	29
7.0.5 LSTM	29
第八章 文本生成	31
8.0.1 Seq2Seq 模型	31
8.0.2 Self-Attention	31
第九章 Transformer	35
9.0.1 位置编码	35
9.0.2 编码器与解码器	36
第二部分 大模型	41
从 Transformer 到智能体	43
第十章 提示词工程	45
10.0.1 什么是提示词工程	45
10.0.2 零样本提示	45
10.0.3 少样本提示	45
10.0.4 思维链提示	46
10.0.5 提示词工程的实践技巧	46
10.0.6 提示词工程的本质	47
第十一章 检索增强生成 (RAG)	49
11.0.1 大语言模型的局限性	49
11.0.2 什么是 RAG	49
11.0.3 文本嵌入	50
11.0.4 向量数据库	50
11.0.5 RAG 的工作流程	51
11.0.6 RAG 的优势	52
11.0.7 RAG 的挑战	52
第十二章 智能体与工具使用	53
12.0.1 从对话到行动	53
12.0.2 什么是智能体	53

目录5

12.0.3 Agent 循环：智能体的心跳

53

12.0.4 工具系统

56

12.0.5 函数调用：连接模型与工具的桥梁

60

12.0.6 系统提示词：智能体的灵魂

61

12.0.7 子智能体：分工协作

63

12.0.8 上下文管理

64

12.0.9 记忆系统

65

12.0.10 编码智能体：Claude Code

66

12.0.11 智能体的挑战与展望

67

第一部分

深度学习

第一章 感知机

1.1 感知机简介

感知机 (Perceptron) 由美国学者 Frank Rosenblatt 于 1957 年提出，作为神经网络最早期的算法之一，感知机奠定了深度学习发展的基础，至今仍具有重要的学习和研究价值。

图 1.1 是一个接收两个输入信号的感知机的例子。感知机接收多个输入信号，并输出一个信号，在感知机模型中，输入与输出的信号通常是二值信号 (0/1)。 x_1 、 x_2 是输入信号， y 是输出信号， w_1 、 w_2 是对应权重。图中的 $\bigcirc$ 代表“神经元”。输入信号被送往神经元时，会被分别乘以固定的权重，神经元会计算传送过来的信号的总和，当这个总和超过了某个阈值 θ ，神经元输出 1，这也被称为“神经元被激活”。

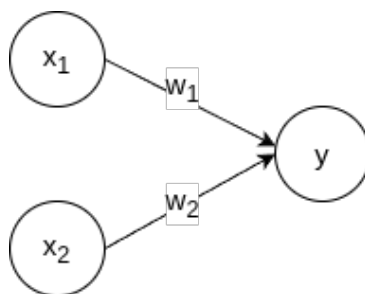


图 1.1: 感知机示例

把上述内容用数学式来表示。

$$y = \begin{cases} 0, & w_1x_1 + w_2x_2 \leq \theta \\ 1, & w_1x_1 + w_2x_2 > \theta \end{cases}$$

1.2 实现与门

下面考虑用感知机来实现与门，表 1.1 是与门的真值表。

x_1	x_2	$y = x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

表 1.1: 与门的真值表

我们需要做的就是确定能满足真值表的 w_1 、 w_2 、 θ 的值。实际上，参数的选择方法有无数多种。比如，当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.7)$ 时，可以满足条件。此外，当 $(w_1, w_2, \theta) = (0.5, 0.5, 0.8)$ 或者 $(1.0, 1.0, 1.0)$ 时，同样也满足与门的条件。设定这样的参数后，仅当 x_1 和 x_2 同时为 1 时，信号的加权总和才会超过给定的阈值 θ 。

为了数学表达的简洁性，我们先把上面感知机实现的数学式子改写一下，用偏置 b 代替阈值 θ ，注意 $b = -\theta$ 。

$$y = \begin{cases} 0, & b + w_1x_1 + w_2x_2 \leq 0 \\ 1, & b + w_1x_1 + w_2x_2 > 0 \end{cases}$$

1.3 局限性

前面我们用感知机实现了与门，现在，我们来探讨一个更具挑战性的例子——异或门。表 1.2 是异或门的真值表，应该设定什么样的参数能满足这个真值表呢？

x_1	x_2	$y = x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

表 1.2: 异或门的真值表

实际上，用前面介绍的感知机是无法实现这个异或门的。思考感知机如何实现与门的，其实是感知机生成一条直线来划分空间，以我们之前实现的与门为例，生成的直线为 $-0.8 + 0.5x_1 + 0.5x_2 = 0$ 。

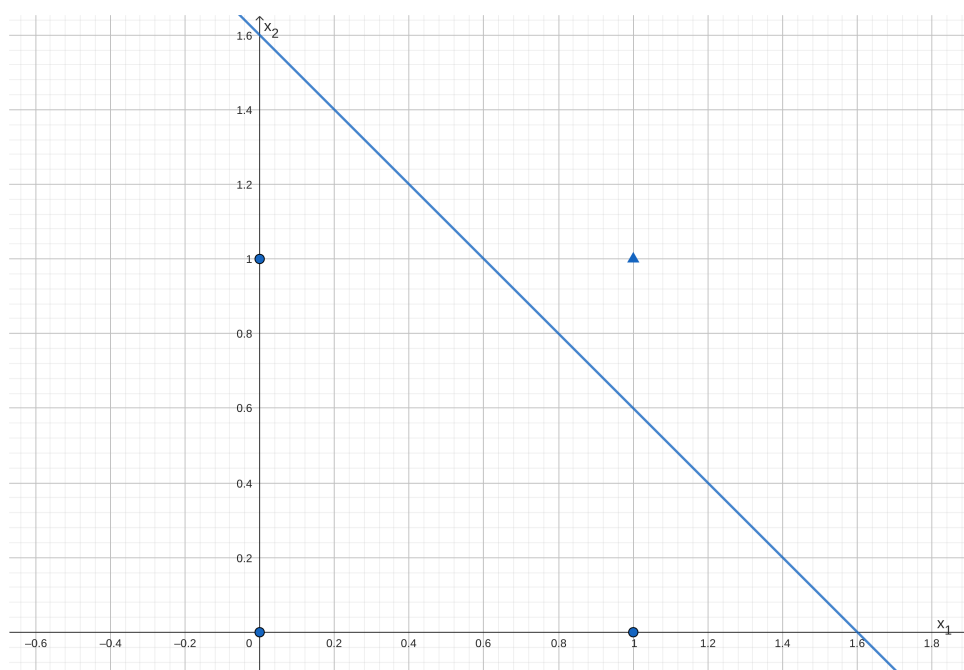


图 1.2: 与门可视化

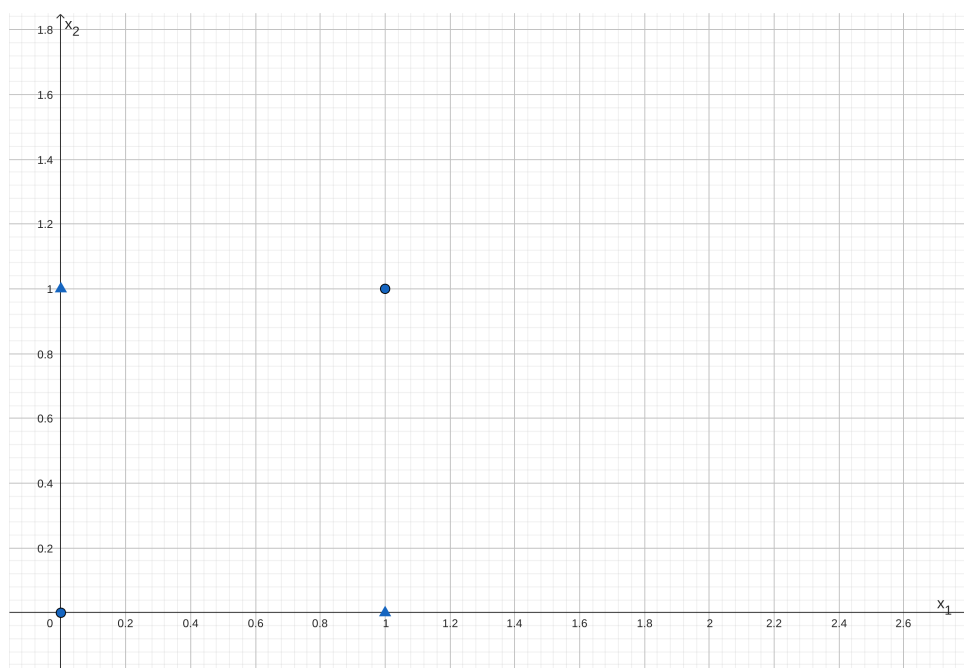


图 1.3: 异或门可视化

想要用一条直线将图 1.3 中的数据分开, 是做不到的。感知机只能表示线性可分的空间, 不过, 就如同我们能通过与门、与非门、或门构建异或门, 我们也能够通过“多层叠加”感知机来处理非

线性可分的问题。

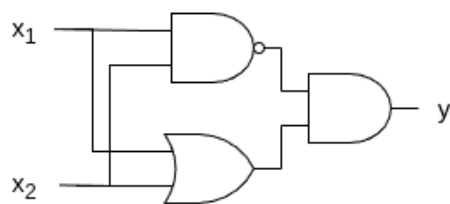


图 1.4: 通过组合实现异或门

第二章 神经网络

2.1 神经网络简介

在前面的感知机模型中，参数的确定并不是由计算机自动完成的，而是由我们人为地根据真值表等“训练数据”手动计算得出。为了让计算机能够自动学习和调整参数，神经网络的概念应运而生。

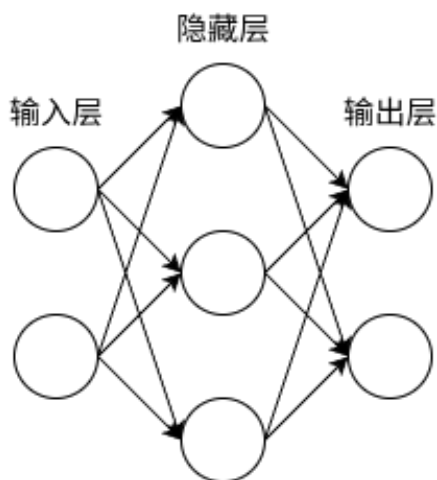


图 2.1: 神经网络示例

将之前感知机的函数改写成以下更简洁的形式。

$$y = h(b + w_1x_1 + w_2x_2) \quad h(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

$h(x)$ 函数会将输入信号的总和转换为输出信号，被称为激活函数 (activation function)。我们之前使用的激活函数是“阶跃函数”，一旦输入超过阈值，就切换输出。除了这个外，神经网络中常用的激活函数有 sigmoid 函数、ReLU (Rectified Linear Unit) 函数。

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{ReLU}(x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$

对于输出层，我们有时不是直接把输出层的值当作最终结果，比如如果需要做个多分类问题，可能需要用 softmax 函数对输出层处理。

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

2.2 损失函数

神经网络“从数据中学习”，由数据自动决定权重参数的值。为了达成这个目的，我们先要决定损失函数 (loss function)，之后，神经网络就以这个指标为基准，寻找最优权重参数。常用的损失函数有均方误差 (mean squared error) 和交叉熵误差 (cross entropy error)。

$$\text{MSE} = \frac{1}{2} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad \text{CE} = - \sum_{i=1}^K \hat{y}_i \log(y_i)$$

这里， y_i 是表示神经网络的输出， $\hat{y}_i$ 表示监督数据， i 表示数据的维数。交叉熵误差主要用于多分类问题，实际上他只计算对应正确解标签的输出，假设正确解标签的索引是“1”，与之对应的神经网络的输出是 0.7，则交叉熵误差是 $-\log 0.7$ 。

使用数据进行学习，就是针对训练数据计算损失函数的值，找出使该值尽可能小的参数。如果遇到大数据，数据量会有几百万、几千万之多，这种情况下以全部数据为对象计算损失函数是不现实的。因此，我们从全部数据中选出一部分，作为全部数据的“近似”。从训练数据中选出一批数据 (mini-batch)，然后对每个 mini-batch 进行学习，这种学习方式称为 mini-batch 学习。

学习的目标是最小化损失函数，梯度法是实现这一目标的常用优化方法：

$$\begin{aligned} x_1^{(t+1)} &= x_1^{(t)} - \eta \frac{\partial f}{\partial x_1}(x_1^{(t)}, x_2^{(t)}) \\ x_2^{(t+1)} &= x_2^{(t)} - \eta \frac{\partial f}{\partial x_2}(x_1^{(t)}, x_2^{(t)}) \end{aligned}$$

η 被称为学习率 (learning rate)。可以通过数值微分 (Numerical Differentiation) 来计算导数，即向 h 中赋入一个微小值，直接计算导数式。

$$\frac{d}{dx} f(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

第三章 计算图

3.1 计算图简介

计算图 (Computational Graph) 是一种将复杂的数学表达式分解为基本运算单元并用图结构表示的方法。在计算图中，节点 (Node) 表示变量或操作 (如加法、乘法、激活函数等)，边 (Edge) 表示数据在操作之间的流动或依赖关系。

假设小明在超市买了两个 5 元/个的苹果，消费税为 10%，请计算他需要支付的总金额。

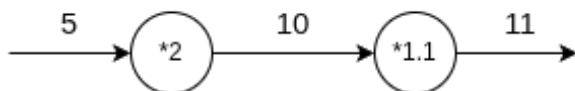


图 3.1: 计算图示例

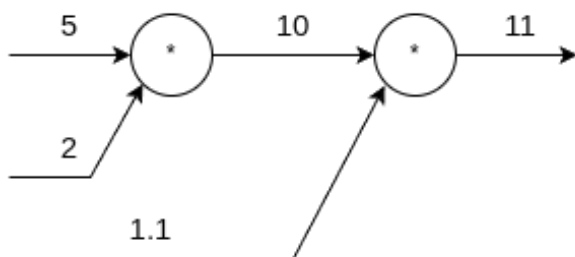


图 3.2: 计算图示例

用计算图解题的情况下，需要先构建计算图，然后在计算图上，从左向右进行计算。这里的“从左向右进行计算”被称为正向传播 (forward propagation)。事实上，也存在反向传播 (backward propagation)，反向传播将在接下来的导数计算中发挥重要作用。

计算图的特征是可以通过传递“局部计算”获得最终结果，这就带来一个优点：无论全局是多么复杂的计算，都可以通过局部计算使各个节点致力于简单的计算，从而简化问题。实际上，使用计算图最大的原因是，可以通过反向传播高效计算导数。

3.2 反向传播

我们再来思考一下之前的问题，我们计算了购买 2 个苹果时加上消费税最终需要支付的金额。这里，假设我们想知道苹果价格的上涨会在多大程度上影响最终的支付金额，即求“支付金额关于苹果的价格的导数”。设苹果的价格为 x ，支付金额为 L ，则相当于求 $\frac{dL}{dx}$ 。这个导数的值表示当苹果的价格稍微上涨时，支付金额会增加多少。

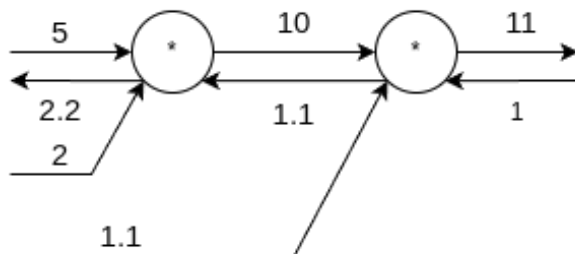


图 3.3: 反向传播示例

反向传播传递“局部导数”，将导数的值写在箭头的下方。在这个例子中，反向传播从右向左传递导数的值 ($1 \rightarrow 1.1 \rightarrow 2.2$)。从这个结果中可知，“支付金额关于苹果的价格的导数”的值是 2.2。这意味着，如果苹果的价格上涨 1 元，最终的支付金额会增加 2.2 元。可以看出，计算图的优点是，可以通过反向传播高效地计算各个变量的导数值。

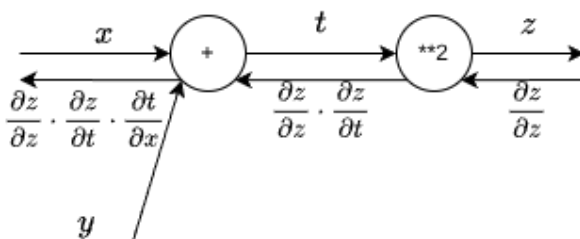


图 3.4: 反向传播示例

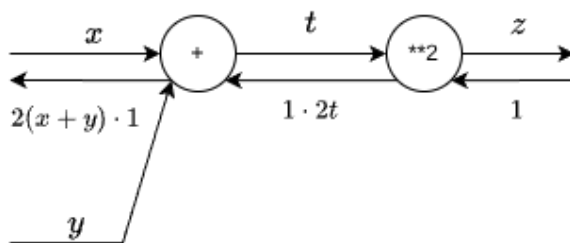


图 3.5: 反向传播示例

计算图的反向传播是基于链式法则成立的，通过提前计算每个节点相对输入的偏导数，可以快速进行反向传播。

第四章 CNN

4.1 CNN 简介

本章的主题是卷积神经网络 (Convolutional Neural Network, CNN), 之前介绍的神经网络中, 相邻层的所有神经元之间都有连接, 这称为全连接网络 (fully-connected Network)。

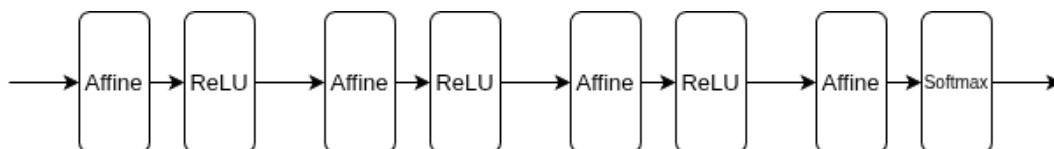


图 4.1: 全连接网络示例

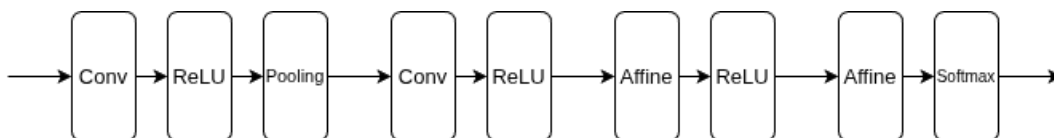


图 4.2: 卷积神经网络示例

相较全连接网络, CNN 中新出现了卷积层 (Convolution 层) 和池化层 (Pooling 层), CNN 的层的连接顺序是 “Conv - ReLU - Pooling” (Pooling 层有时会被省略)。还需要注意的是, 在 CNN 中, 靠近输出的层中使用了之前的 “Affine - ReLU” 组合。此外, 最后的输出层中使用了之前的 “Affine - Softmax” 组合, 这些都是 CNN 中比较常见的结构。

4.2 卷积层

全连接层 (Affine 层) 存在什么问题呢? 那就是数据的形状被 “忽视” 了。比如, 输入数据是图像时, 图像通常是高、长、rgb 通道方向上的 3 维形状。但是, 向全连接层输入时, 需要将 3 维数据拉平为 1 维数据。图像是 3 维形状, 这个形状中含有重要的空间信息。比如, 空间上邻近的像素为相似的值, 形状中可能隐藏有值得提取的特征。

卷积层可以保持形状不变。当输入数据是图像时, 卷积层会以 3 维数据的形式接收输入数据, 并同样以 3 维数据的形式输出至下一层。利用卷积层, 有可能正确理解输入数据的形状的信息, 卷

卷积层的输入输出数据称为特征图 (feature map)。其中，输入数据称为输入特征图 (input feature map)，输出数据称为输出特征图 (output feature map)。

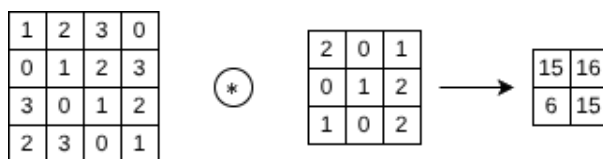


图 4.3: 卷积运算示例

卷积层进行的处理就是卷积运算，相当于图像处理中的“滤波器运算”，滤波器也被称为卷积核，卷积运算以一定间隔滑动滤波器的窗口并应用，这个计算也被称为乘积累加运算。

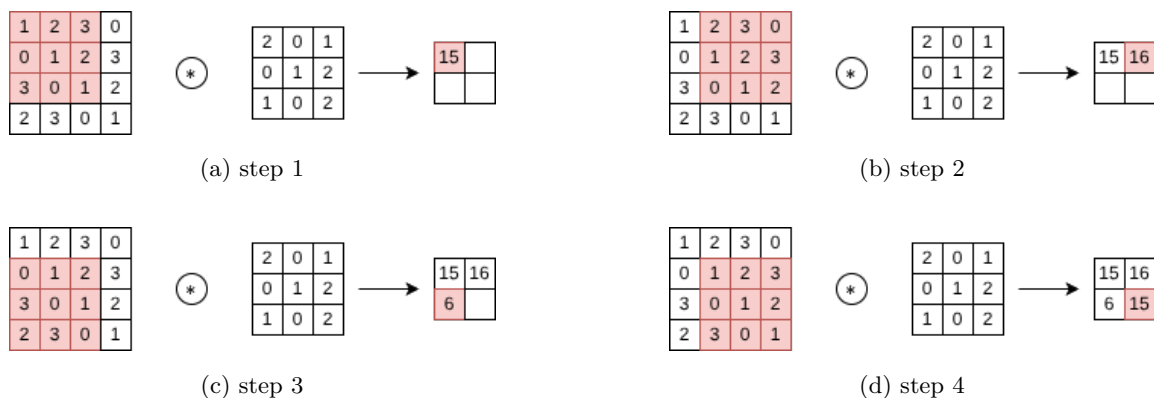


图 4.4: 卷积运算流程说明

CNN 中也存在偏置，偏置通常只有 1 个，这个值会被加到应用了滤波器的所有元素上。

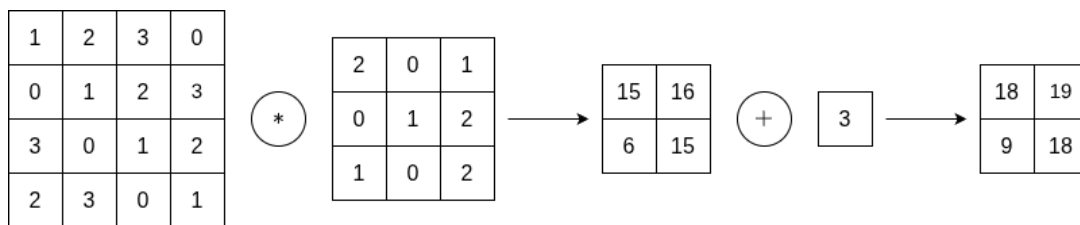


图 4.5: 偏置示例

在进行卷积层的处理之前，有时要向输入数据的周围填入固定的数据（比如 0），这称为填充 (padding)，主要是为了调整输出的大小。如下图所示，通过应用了幅度为 1 的填充，大小为 (4, 4) 的输入数据变成了 (6, 6) 的形状，然后，应用大小为 (3, 3) 的滤波器，生成了大小为 (4, 4) 的输出数据。

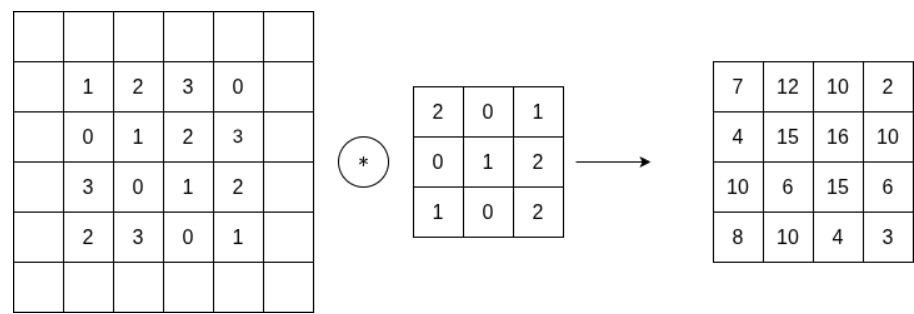


图 4.6: 填充示例

应用滤波器的位置间隔称为步幅（stride）。下图为一个步幅为 2 卷积运算示例。

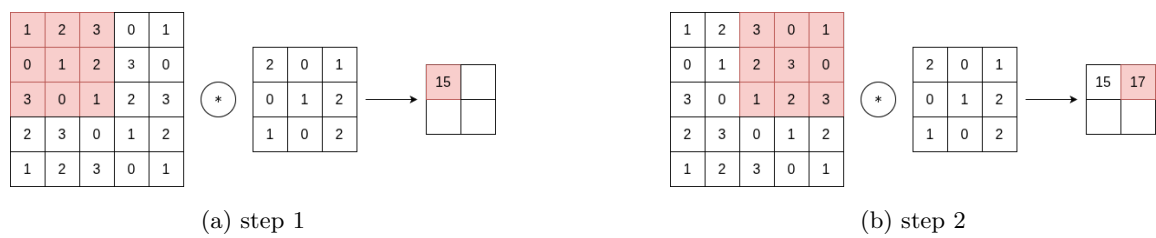


图 4.7: 步幅示例

图像是 3 维数据，除了高、长方向之外，还需要处理通道方向，在 3 维数据的卷积运算中，输入数据和滤波器的通道数要设为相同的值。最后数据输出是 1 张特征图，如果要数据输出在通道方向上也拥有维度，那么就需要用到多个滤波器，生成多个特征图。

4.3 池化层

池化是缩小高、长方向上的空间的运算，下图是按步幅 2 进行 2×2 的 Max 池化时的处理顺序。一般来说，池化的窗口大小会和步幅设定成相同的值。除了 Max 池化之外，还有 Average 池化、随机池化等，在图像识别领域，主要使用 Max 池化。

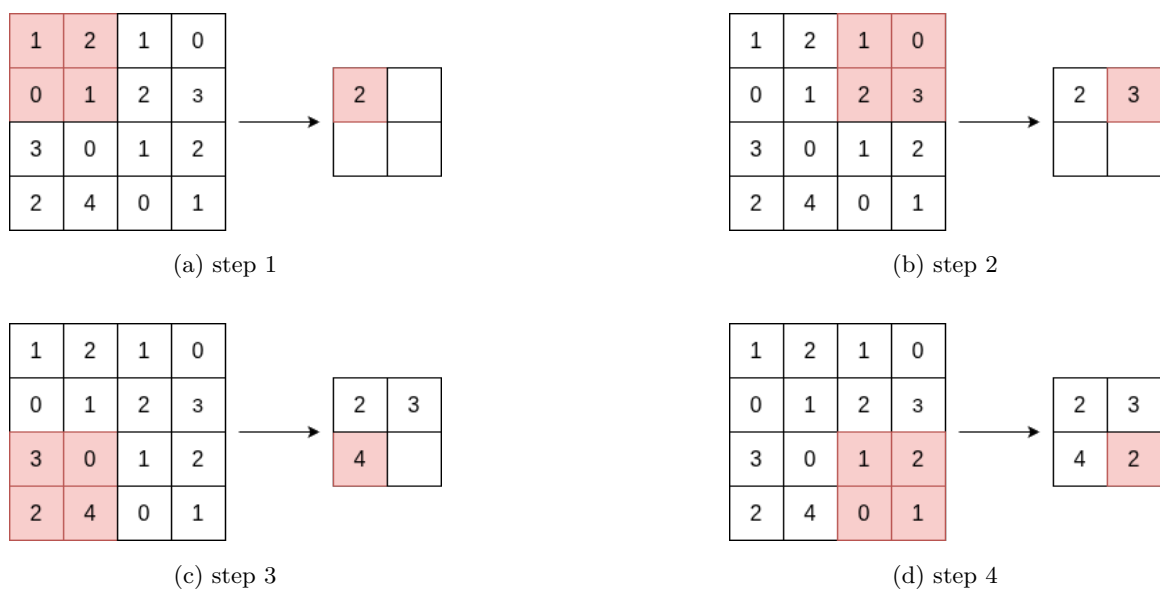


图 4.8: 池化层流程说明

池化层的特征

- 没有要学习的参数: 池化层和卷积层不同, 没有要学习的参数。
- 通道数不发生变化: 经过池化运算, 输入数据和输出数据的通道数不会发生变化。
- 对微小的位置变化具有鲁棒性: 输入数据发生微小偏差时, 池化仍会返回相同的结果。

第五章 自然语言处理

5.1 单词表示

自然语言处理 (Natural Language Processing, NLP), 它的目标就是让计算机理解人说的话, 进而帮助人们更好地完成某些事情。语言是由文字构成的, 而语言的含义是由单词构成的, 即单词是含义的最小单位。所以, 为了让计算机理解自然语言, 第一步就是让它理解单词含义。

5.2 基于同义词词典的方法

要表示单词含义, 首先可以考虑通过人工方式来定义单词含义, 一种方式就是像词典一样, 一个词一个词地说明单词含义。NLP 中广泛运用的一种词典是同义词词典 (thesaurus), 在词典中, 含义类似的单词被归类到同一个组中。比如, car 的同义词有 automobile、motorcar 等。另外, 在同义词词典中, 有时还会定义更多的关系, 比如上位下位关系、整体部分关系等。在 NLP 领域, 最著名的同义词词典是普林斯顿大学于 1985 年开始开发的同义词词典 WordNet。

同义词词典有许多缺陷, 第一点是它难以实时更新, 随着时间的推移, 会有新词不断出现, 也会有旧词被赋予新的含义, 同义词词典很难反映出这种变化。第二点是生成同义词词典的人力成本高, 以英文为例, 据说现有的英文单词总数超过 1000 万个, 光靠人力创造完全的同义词词典不太现实。第三点是它无法表示单词的微妙差异, 含义相近的单词之间可能也会有细微的区别, 这些区别同义词词典不能很好体现。

5.3 基于统计的方法

基于统计的方法的目标是将单词表示成向量形式, 在 NLP 中也称为分布式表示, 这个想法基于分布式假设 (distributional hypothesis): “某个单词的含义由它周围的单词形成”, 也就是说单词含义可以它所在的上下文表示。该方法需要使用语料库 (corpus), 即大量的文本数据, 如 Wikipedia 等。考虑句子: “I say yes and you say no.”, 首先对文本进行分词, 记录下每个单词的上下文 (假设窗口大小为 1), I 的上下文只有 say, 其表示为:

	I	say	yes	and	you	no	.
I	0	1	0	0	0	0	0

即 you 向量表示为 (0, 1, 0, 0, 0, 0, 0), 继续处理剩余单词。最后生成一个生成共现矩阵 (co-occurrence matrix), 矩阵每行对应一个单词向量:

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

在上面的矩阵中, 元素表示两个单词同时出现的次数, 这种表示有时可能出现负值。考虑语料库频繁出现 “...the car...” 的情况, 如果只看单词的出现次数, 有可能出现 the 和 car 的相关性比 car 与 drive 相关性更强的情况。为了解决这一问题, 可以使用点互信息 (Pointwise Mutual Information, PMI) 指标, 对于随机变量 x 和 y , 它们的 PMI 定义如下:

$$\text{PMI}(x, y) = \log \frac{P(x, y)}{P(x)P(y)}$$

为了避免 $\log 0$, 实践上会使用正的点互信息 (Positive PMI, PPMI) :

$$\text{PPMI}(x, y) = \max(0, \text{PMI}(x, y))$$

5.4 基于推理的方法

基于统计的方法在处理大规模语料库会出现问题, 由于单词数量非常大, 生成的矩阵也会非常庞大, 处理起来不太方便。本节介绍基于推理的方法, 他的原理相当于训练一个嵌入模型, 让它做完形填空, 通过反复的预测训练, 学习到单词的嵌入表示。

在使用神经网络处理单词之前, 首先需要将单词转换为向量表示。其中最简单的一种方式是使用 one-hot 编码。这种表示方法中, 向量的长度等于词汇表的大小, 且仅将对应单词 ID 的位置设为 1, 其余位置均为 0。

Word2Vec 是 Google 于 2013 开源的词向量计算工具, 其中使用的模型之一为 CBOW (continuous bag-of-words) 模型。CBOW 模型的输入是上下文, 这个上下文用单词列表表示, 每个单词用 one-hot 表示成向量, 因此如果上下文考虑 N 个单词, 则输入层会有 N 个。CBOW 中, 中间层的神经元是各个输入层经全连接层变换后得到的值的平均, 输出层神经元数目等于词表大小, 这些神经元是各个单词的得分, 它的值越大, 说明对应单词的出现概率就越高。

Word2Vec 中还有一个被称为 skip-gram 的模型, skip-gram 反转了 CBOW 模型的目标, CBOW 模型从上下文预测单词, 而 skip-gram 模型则从单词预测上下文。在 Word2Vec 中, 模型的输入侧的中间层权重就是单词的分布式表示。

考虑 one-hot 表示, 由于其中只有一个为 1 的元素, 它与中间层权重矩阵的乘积其实就是取矩阵的某一行, 也即获得单词的分布式表示, 这个分布式表示也被称为词嵌入 (word embedding), 所

以输入侧中间层有时也被称为 Embedding 层。由于这个操作比较简单，实践上一般用查表的方式代替代替矩阵乘积。

第六章 分词

6.1 分词简介

要对文本进行处理，第一件事就是分词（Tokenization），将文本切分成一个个 Token，Token 可以是单词、子词或字符等。分词的方式有很多种，比较简单的方式是按字、按词切分，复杂一点的方式是 n-gram 分词，切分时 n 个 Token 为一组来对文本进行切分。

- 文本：人工智能让世界变得更美好
- 按字：人 工 智 能 让 世 界 变 得 更 美 好
- 按词：人 工 智 能 让 世 界 变 得 更 美 好
- 按字 Bi-Gram：人/工 工/智 智/能 能/让 让/世 世/界 界/变 变/得 得/更 更/美 美/好 好/
- 按词 Bi-Gram：人工智能/让 让/世界 世界/变得 变得/更 更/美好 美好/

由于分词是按词表进行的，词表的产生就是一个重要的工作，由此产生了多种分词算法，常见的有 BPE、BBPE、WordPiece 等。

6.2 BPE

先介绍 BPE（byte pair encoding）的工作原理，假设有一个语料库，我们首先从其中提取所有的单词并计算它们出现的次数。假设提取出来的单词和计数是：

(cost, 2)、(best, 2)、(menu, 1)、(men, 1)、(camel, 1)

先将所有的单词变成字符序列，用出现过的所有字符初始化词表，此时词表的大小为 11。

a, b, c, e, l, m, n, o, s, t, u

接下来，我们需要定义词表迭代的终止条件，假设我们要创建大小为 14 的词表。为了在词表中添加一个新 Token，我们需要先确定出现得最频繁的符号对，将它们合并之后添加到词表中，重复这一步骤，直到达到词表的大小要求。

延续前面的例子，从字符序列可以看出，出现得最频繁的符号对是 s 和 t，符号对 s 和 t 出现了 4 次（两次是在 cost 中，另外两次是在 best 中），因此将 st 添加到词表。

a, b, c, e, l, m, n, o, s, t, u, st

下面，重复同样的步骤，再次检查出现的最频繁的符号对，可以计算出现在出现得最频繁的符号对是 m 和 e，这个符号对出现了 3 次（menu、men、camel）。将 m 和 e 合并，并将其添加到词表中，继续以上步骤，直到词表大小为 14，最终词表如下所示：

a, b, c, e, l, m, n, o, s, t, u, st, me, men
--

BPE 所涉及的步骤小结如下：

1. 从给定的语料集中提取单词并计算它们出现的次数。
2. 确定词表的大小。
3. 将单词拆分成一个字符序列。
4. 将字符序列中的所有非重复字符添加到词表中。
5. 选择并合并具有高频率的符号对。
6. 重复步骤 5，直到达到步骤 2 中所设定的词表大小。

6.3 BBPE

BBPE (byte-level byte pair encoding, BBPE) 的工作原理与 BPE 非常相似，但它不使用字符序列，而是使用字节序列。假设输入文本只有单词 best，在 BPE 中，需要将单词转换为字符序列：b e s t，而在 BBPE 中，我们不是将单词转换为字符序列，而是将其转换为字节序列：62 65 73 74，假设输入是“你好”这个中文词组，则会被转换为字节序列：e4 bd a0 e5 a5 bd。BBPE 的好处是字节表示的通用性使其能处理任何编码的文本。

6.4 WordPiece

WordPiece 和 BPE 主要的不同之处在于合并对的选择方式。WordPiece 不是选择频率最高的字符对，而是对每个字符对计算一个得分，使用公式：

$$\text{score} = \frac{\text{freq_of_pair}}{\text{freq_of_first_element} \times \text{freq_of_second_element}}$$

第七章 RNN

7.1 语言模型

再考虑一下 word2vec 的 CBOW 模型，假设单词序列为 $w_1, w_2, \dots, w_T$ 且窗口大小为 1，它所做的事情就是从上下文 w_{t-1} 和 w_{t+1} 预测目标词 w_t 。当给定上下文时目标词是 w_t 的概率为 $P(w_t | w_{t-1}, w_{t+1})$ ，CBOW 模型就是对这一后验概率进行建模，如果使用交叉熵误差，其损失函数是 $L = -\log P(w_t | w_{t-2}, w_{t-1})$ 。在上一章我们使用 CBOW 模型的权重获得单词的表示，那它本来的目的“从上下文预测目标词”是否能用来做点事情呢？这就得提到语言模型了。

语言模型 (language model) 使用概率来评估一个单词序列发生的可能性。比如，对于 “you say goodbye” 这一单词序列，语言模型给出高概率；对于 “you say good die” 这一单词序列，模型则给出低概率。典型的语言模型应用有机器翻译和语音识别，比如，语音识别系统会根据人的发言生成多个句子作为候选，此时，使用语言模型，可以按照单词序列发生的可能性对候选句子进行排序。用数学表示语言模型：

$$\begin{aligned} P(w_1, \dots, w_m) &= P(w_m | w_1, \dots, w_{m-1}) \cdot P(w_{m-1} | w_1, \dots, w_{m-2}) \cdots P(w_2 | w_1) \cdot P(w_1) \\ &= \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \end{aligned}$$

假设我们将 CBOW 模型改造成语言模型，那么首先得将原来对称的上下文改成只有目标词左侧的上下文，这样可以近似实现语言模型：

$$P(w_1, \dots, w_m) = \prod_{t=1}^m P(w_t | w_1, \dots, w_{t-1}) \approx \prod_{t=1}^m P(w_t | w_{t-2}, w_{t-1})$$

不过这个实现还是有点问题，首先是它只能处理固定长度的上下文，会有信息丢失；其次是上下文的单词顺序会被忽视，比如，(you, say) 和 (say, you) 会被作为相同的内容进行处理。

7.2 RNN 介绍

循环神经网络 (Recurrent Neural Network, RNN) 就可以很好地解决上述问题。之前我们看到的神经网络都是前馈型神经网络，网络的传播方向是单向的，虽然前馈网络结构简单、易于理解，但它不能很好地处理时间序列数据。

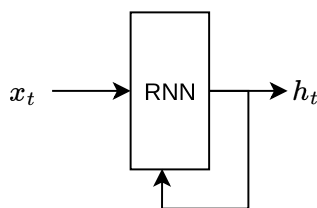


图 7.1: RNN 层示例

RNN 层有环路，通过该环路，数据可以在层内循环。时序数据 $(x_0, x_1, \dots, x_t, \dots)$ 被输入到层中。然后，以与输入对应的形式，输出 $(h_0, h_1, \dots, h_t, \dots)$ 。RNN 有两个权重，分别是将输入 x 转化为输出 h 的权重 W_x 和将前一个 RNN 层的输出转化为当前时刻的输出的权重 W_h 。RNN 的计算用数学表示为： $h_t = \tanh(h_{t-1}W_h + x_tW_x + b)$ 。RNN 的输出 h_t 称为隐藏状态 (hidden state) 或隐藏状态向量 (hidden state vector)。

由于 RNN 层有环路，因此不能直接使用前馈神经网络的反向传播算法，RNN 的方向传播是基于时间的反向传播 (Back Propagation Through Time, BPTT)，具体做法是展开 RNN 的循环，把它看作普通的神经网络，然后运用之前的反向传播方法即可。

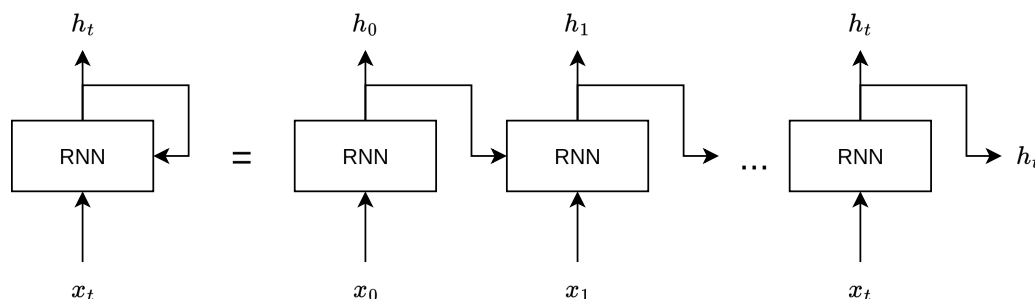


图 7.2: RNN 的展开

在处理长度为 1000 的时序数据时，如果展开 RNN 层，它将成为在水平方向上排列有 1000 个层的网络，序列太长的话，会出现计算量或者内存使用量方面的问题，此外，随着层变长，梯度逐渐变小，梯度将无法向前一层传递。因此，考虑将时间轴方向上过长的网络在合适的位置进行截断，从而创建多个小型网络，然后对截出来的小型网络执行误差反向传播法，这个方法称为 Truncated BPTT。

在 RNN 的学习中，为了执行 mini-batch 学习，需要将输入数据进行“偏移”。对长度为 1000 的时序数据，如果将批大小设为 2，那么第 1 笔样本数据从头开始按顺序输入，第 2 笔数据就从第 500 个数据开始按顺序输入。

7.3 语言模型的评估

困惑度 (perplexity) 常被用作评价语言模型的预测性能的指标，困惑度表示“概率的倒数”。考虑 “you say goodbye and i say hello.” 语料库，假设在向语言模型传入单词 you 时会输出的概率分布中，单词是 say 的概率为 0.2，那么困惑度就是 $1/0.2=5$ 。困惑度可以解释为下一个可以选择的选项的数量，在刚才的例子中，模型的困惑度是 5，这意味着下一个要出现的单词的候选个数为 5 个。在输出数据为多个的情况下，困惑度计算公式为： $Perplexity = e^L$ ，其中，L 为交叉熵损失函数的值。

7.4 梯度消失和梯度爆炸

前面介绍的简单的 RNN 在无法很好地学习到时序数据的长期依赖关系。RNN 中使用了函数 $\tanh(x)$ ，它的导数为 $1 - \tanh^2(x)$ ，这个值小于 1，并且随着 x 远离 0，它的值在变小，这就导致 RNN 在处理长序列时容易出现梯度消失的问题。另外，RNN 中使用了矩阵乘积，可能导致梯度消失或者梯度爆炸，这里不具体分析了。

梯度爆炸的问题可以通过梯度裁剪 (gradients clipping) 来解决，

$$g \leftarrow \begin{cases} g, & \text{if } \|g\|_2 \leq \tau \\ g \cdot \frac{\tau}{\|g\|_2}, & \text{if } \|g\|_2 > \tau \end{cases}$$

这里假设可以将神经网络用到的所有参数的梯度整合成一个向量 g ，且阈值设置为 τ 。

7.5 LSTM

在 RNN 的学习中，梯度消失也是一个大问题，为了解决这个问题，LSTM (Long Short-Term Memory) 中增加了一种名为“门”的结构，并且 LSTM 中有专门的权重参数用于控制门的开合程度，这些权重参数也会在学习中被更新。在 LSTM 中存在记忆单元 c ，仅在 LSTM 层内部接收和传递数据，不向其他层输出。其中 $h_t = \tanh(c_t)$ 。

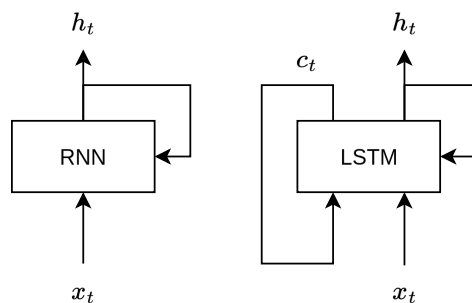


图 7.3: RNN 与 LSTM

输出门 (output gate) 针对 $\tanh(c_t)$ 的元素, 调整它们作为下一时刻的隐藏状态的重要程度。输出门的开合程度根据输入 x_t 和上一个状态 h_{t-1} 求出 (σ 表示 sigmoid 函数), 最后将这个 o_t 和 $\tanh(c_t)$ 的对应元素的乘积作为 h_t 输出。

$$o_t = \sigma(x_t W_x^{(o)} + h_{t-1} W_h^{(o)} + b^{(o)})$$

遗忘门 (forget gate) 是一个忘记不必要记忆的门, 遗忘门的输出 f_t 与记忆单元 c_{t-1} 相乘, 生成新的记忆单元 c_t :

$$f_t = \sigma(x_t W_x^{(f)} + h_{t-1} W_h^{(f)} + b^{(f)})$$

遗忘门用来遗忘记忆, 现在我们还想向记忆单元添加一些应当记住的新信息, 为此我们添加新的 $\tanh$ 节点, 通过将这个 g_t 加到上一时刻的 c_{t-1} 上, 从而形成新的记忆:

$$g_t = \tanh(x_t W_x^{(g)} + h_{t-1} W_h^{(g)} + b^{(g)})$$

最后添加一个输入门 (input gate), 将 i_t 和 g_t 的对应元素的乘积添加到记忆单元中:

$$i_t = \sigma(x_t W_x^{(i)} + h_{t-1} W_h^{(i)} + b^{(i)})$$

因为在 LSTM 中, 记忆单元的更新仅有 + 和 $\times$ 运算, 因此梯度消失和梯度爆炸问题都得到有效解决。

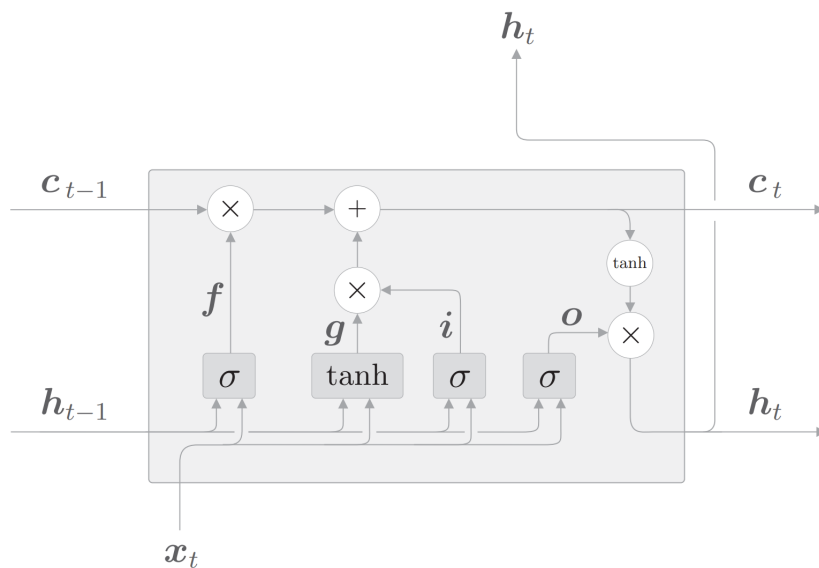


图 7.4: LSTM 的计算过程

第八章 文本生成

8.1 Seq2Seq 模型

语言模型根据已经出现的单词输出下一个出现的单词的概率分布，通过对这个概率分布进行采样，就可以逐步生成完整的文本。最常见的采样方法是每次选择概率最高的词作为下一个输出，重复该过程，直到生成结束符号 $\langle \text{eos} \rangle$ 为止。

Seq2Seq 模型则是一类用于将一种序列转换为另一种序列的模型，广泛应用于机器翻译、语音识别等任务。它通常由两个模块组成：Encoder 和 Decoder，因此也被称为 Encoder-Decoder 模型。编码器将输入序列编码为一个向量表示，常见结构是 Embedding 层加 RNN 层；解码器则根据这个向量表示逐步生成输出序列，其生成过程可以采用前面提到的采样策略。

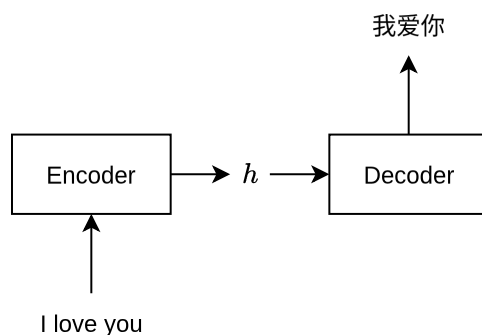


图 8.1: Seq2Seq 模型示例

编码器的输出是固定长度的向量，但输入序列的长度是可变的，这可能会导致部分信息丢失。为了解决这个问题，我们可以使用编码器在每个时间步的隐藏状态，将其拼接成一个矩阵，作为解码器的输入。由于解码器原本只接收一个向量输入，因此也需要相应地进行调整。我们可以通过对该矩阵的行向量进行加权平均来将其转换为向量，而加权平均中权重的确定，就是注意力机制（Attention Mechanism）所解决的问题。

8.2 Self-Attention

在基于 RNN 的 Seq2Seq 模型中，注意力机制使得解码器在生成每个输出单词时，能够动态地关注输入序列中的不同部分。例如，在将 “I love you” 翻译为 “我爱你” 的过程中，生成 “我” 时模型更倾向于关注 “I”，而生成 “爱” 时则更多关注 “love”。既然输出序列可以使用注意力机制，那么输入序列是否也可以使用注意力机制呢？

答案是肯定的，这正是 Transformer 模型中自注意力机制 (Self-Attention) 的作用。需要说明的是，Transformer 作为 Seq2Seq 模型的一种实现方式，也包含类似于基于 RNN 的 Seq2Seq 模型中的注意力机制，这部分在 Transformer 中被称为交叉注意力 (Cross-Attention)。让我们通过一个例子来快速理解自注意力机制，例句：

A dog ate the food because it was hungry

显然，例句中的代词 it 指代的是 dog 而不是 food，自注意力机制的作用就是关注句子间单词的关系。

将句子输入模型的编码器，得到一个输出矩阵 $\mathbf{X}$ ，为了计算自注意力，我们需要再创建三个新的矩阵：查询 (query) 矩阵 $\mathbf{Q}$ 、键 (key) 矩阵 $\mathbf{K}$ ，以及值 (value) 矩阵 $\mathbf{V}$ 。为了创建它们，我们需要先创建另外三个权重 (weight) 矩阵， $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ 。用矩阵 $\mathbf{X}$ 分别乘以矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 、 $\mathbf{W}^V$ ，依次创建出 $\mathbf{Q}$ 、 $\mathbf{K}$ 和 $\mathbf{V}$ 。值得注意的是，权重矩阵 $\mathbf{W}^Q$ 、 $\mathbf{W}^K$ 和 $\mathbf{W}^V$ 的初始值完全是随机的，但最优值则需要通过训练获得。我们取得的权值越优，通过计算所得的查询矩阵、键矩阵和值矩阵也会越精确。

假设输入句子为 “I am good”，自注意力机制首先要计算查询矩阵 $\mathbf{Q}$ 与键矩阵 $\mathbf{K}^T$ 的点积。

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \mathbf{q}_1 \cdot \mathbf{k}_3 \\ \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \mathbf{q}_2 \cdot \mathbf{k}_3 \\ \mathbf{q}_3 \cdot \mathbf{k}_1 & \mathbf{q}_3 \cdot \mathbf{k}_2 & \mathbf{q}_3 \cdot \mathbf{k}_3 \end{bmatrix}$$

首先，来看 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵的第一行，可以看到，这一行计算的是查询向量 $\mathbf{q}_1$ (I) 与所有的键向量 $\mathbf{k}_1$ (I)、 $\mathbf{k}_2$ (am) 和 $\mathbf{k}_3$ (good) 的点积。通过计算两个向量的点积可以知道它们之间的相似度。因此，通过计算查询向量 ($\mathbf{q}_1$) 和键向量 ($\mathbf{k}_1$ 、 $\mathbf{k}_2$ 、 $\mathbf{k}_3$) 的点积，可以了解单词 I 与句子中的所有单词的相似度。我们了解到，I 这个词与自己的关系比与 am 和 good 这两个词的关系更紧密，因为点积值 110 大于 90 和 80。

$$\mathbf{Q} \cdot \mathbf{K}^T = \begin{bmatrix} 110 & 90 & 80 \\ 70 & 99 & 70 \\ 90 & 70 & 100 \end{bmatrix}$$

表 8.1: 点积示例，本节的结果均为假设值，后续不再说明

自注意力机制的第 2 步是将 $\mathbf{Q} \cdot \mathbf{K}^T$ 矩阵除以键向量维度的平方根，这样做的目的主要是获得稳定的梯度。

目前所得的相似度分数尚未被归一化，所以第 3 步是使用 softmax 函数对其进行归一化处理。应用 softmax 函数将使数值分布在 0 到 1 的范围内，且每一行的所有数之和等于 1。这个矩阵被称为分数 (score) 矩阵，通过它，我们可以了解句子中的每个词与所有词的相关程度。比如：I 这个词与它本身的相关程度是 90%，与 am 这个词的相关程度是 7%，与 good 这个词的相关程度是 3%。

$$\text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}}\right) = \begin{bmatrix} & \text{I} & \text{am} & \text{good} \\ \text{I} & 0.90 & 0.07 & 0.03 \\ \text{am} & 0.025 & 0.95 & 0.025 \\ \text{good} & 0.21 & 0.03 & 0.76 \end{bmatrix}$$

自注意力机制的最后一步是计算注意力矩阵 $\mathbf{Z}$ ，注意力矩阵包含句子中每个单词的注意力值。它通过将分数矩阵乘以值矩阵 $\mathbf{V}$ 得出。

$$\begin{aligned} \mathbf{Z} &= \text{softmax}\left(\frac{\mathbf{Q} \cdot \mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \\ &= \begin{bmatrix} 0.90 & 0.07 & 0.03 \\ 0.025 & 0.95 & 0.025 \\ 0.21 & 0.03 & 0.76 \end{bmatrix} \begin{bmatrix} 67.85 & 91.2 & \dots & 0.13 \\ 13.13 & 63.1 & \dots & 4.44 \\ 12.12 & 96.1 & \dots & 43.4 \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{z}_1 \\ \mathbf{z}_2 \\ \mathbf{z}_3 \end{bmatrix} \end{aligned}$$

可以看出，单词 I 的自注意力值 $\mathbf{z}_1$ 是分数加权的值向量之和。所以， $\mathbf{z}_1$ 的值将包含 90% 的值向量 $\mathbf{v}_1$ (I)、7% 的值向量 $\mathbf{v}_2$ (am)，以及 3% 的值向量 $\mathbf{v}_3$ (good)。

$$\mathbf{z}_1 = 0.90 \begin{bmatrix} 67.85 & 91.2 & \dots \end{bmatrix} + 0.07 \begin{bmatrix} 13.13 & 63.1 & \dots \end{bmatrix} + 0.03 \begin{bmatrix} 12.12 & 96.1 & \dots \end{bmatrix}$$

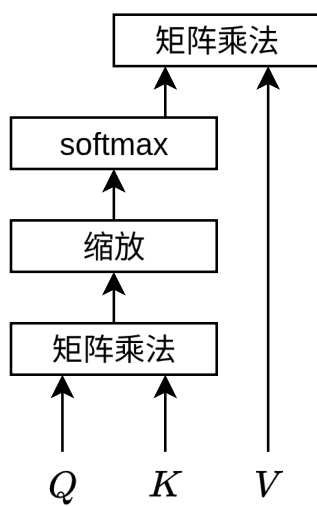


图 8.2: 自注意力计算流程

第九章 Transformer

9.1 位置编码

在 RNN 中，句子是逐词送入网络的，以 I am good 为例，首先把 I 作为输入，接下来是 am，最后是 good。然而，Transformer 并不遵循这个模式，它将句子中的所有词并行地输入到神经网络中。由于是并行输入，所以词序需要通过其他方式表示，Transformer 所采用的方式是位置编码。

在 Transformer 中，位置编码以矩阵的形式表示，位置编码矩阵 $\mathbf{P}$ 的维度与输入矩阵 $\mathbf{X}$ 的维度相同。在将输入矩阵直接传给 Transformer 之前，我们需将位置编码矩阵 $\mathbf{P}$ 加到输入矩阵 $\mathbf{X}$ 中，再将其作为输入送入网络。Transformer 中使用了正弦函数来计算位置编码：

$$P(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right) \quad P(\text{pos}, 2i + 1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

在上面的等式中，pos 表示该词在句子中的位置， i 表示在输入矩阵中的位置。

$$\mathbf{P} = \begin{bmatrix} \sin\left(\frac{\text{pos}}{10000^0}\right) & \cos\left(\frac{\text{pos}}{10000^0}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^1}\right) & \cos\left(\frac{\text{pos}}{10000^1}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \\ \sin\left(\frac{\text{pos}}{10000^2}\right) & \cos\left(\frac{\text{pos}}{10000^2}\right) & \sin\left(\frac{\text{pos}}{10000^{2/4}}\right) & \cos\left(\frac{\text{pos}}{10000^{2/4}}\right) \end{bmatrix}$$

9.2 编码器与解码器

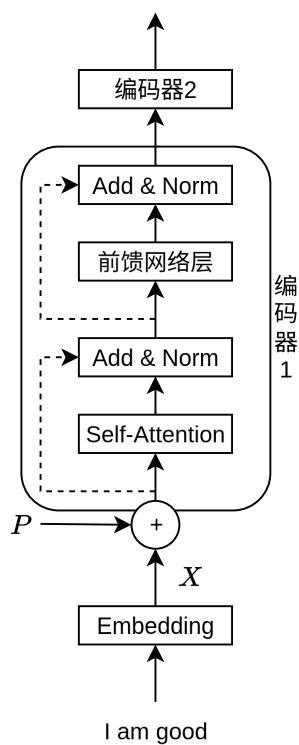


图 9.1: Transformer 编码器结构

Transformer 的编码器由一组 N 个编码器串联而成，每一个编码器的构造都是相同的，由自注意力层和前馈网络层组成，每层的输出都经过残差连接和归一化层（Add & Norm）。前馈网络由多个全连接层组成，一般采用 ReLU 激活函数。

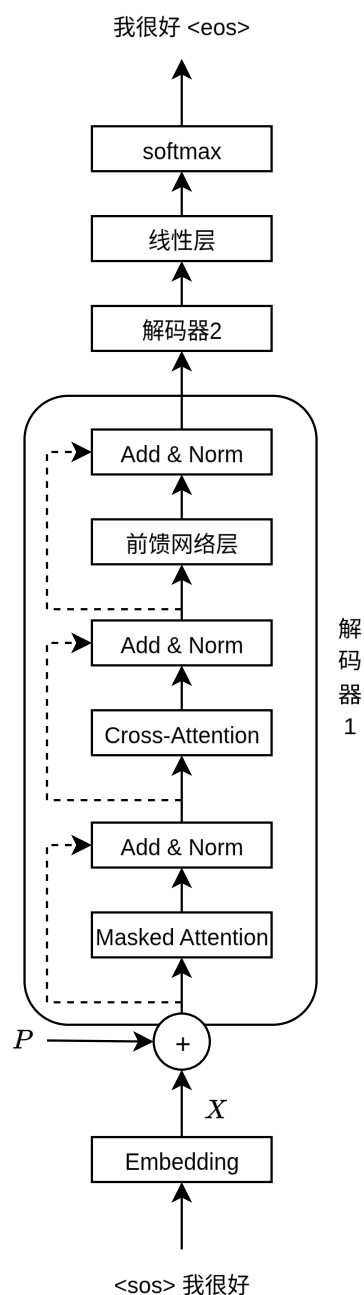


图 9.2: Transformer 解码器结构

相较编码器，解码器的注意力层不太一样，它需要关注编码器的输出，因此解码器的注意力层分为两部分：带掩码的注意力层和交叉注意力层。交叉注意力层都有两个输入：一个来自带掩码的注意力层，另一个是编码器的输出。让我们用 R 来表示编码器的输出值，用 M 来表示由带掩码的注意力层输出的注意力矩阵，注意力的计算使用矩阵 M 创建查询矩阵 Q ，使用 R 创建键矩阵和值矩阵。在解码器中的带掩码的注意层中，我们对注意力计算中的矩阵进行掩码处理，确保解码

器在生成每个单词时只能关注之前的单词。

$$\frac{\mathbf{Q}_i \cdot \mathbf{K}_i^T}{\sqrt{d_k}} = \begin{bmatrix} & \text{<sos>} & \text{我} & \text{很} & \text{好} \\ \text{<sos>} & 9.125 & -\infty & -\infty & -\infty \\ \text{我} & 5.0 & 12.37 & -\infty & -\infty \\ \text{很} & 7.25 & 5.0 & 10.37 & -\infty \\ \text{好} & 1.5 & 1.37 & 1.87 & 10.0 \end{bmatrix}$$

获得解码器的输出之后，我们将其送入线性层和 softmax 层，线性层将生成一个 logit 向量，其大小等于原句中的词汇量，然后，将 softmax 函数应用于 logit 向量，从而得到概率分布，最后对其进行采样得到最终的文本输出。

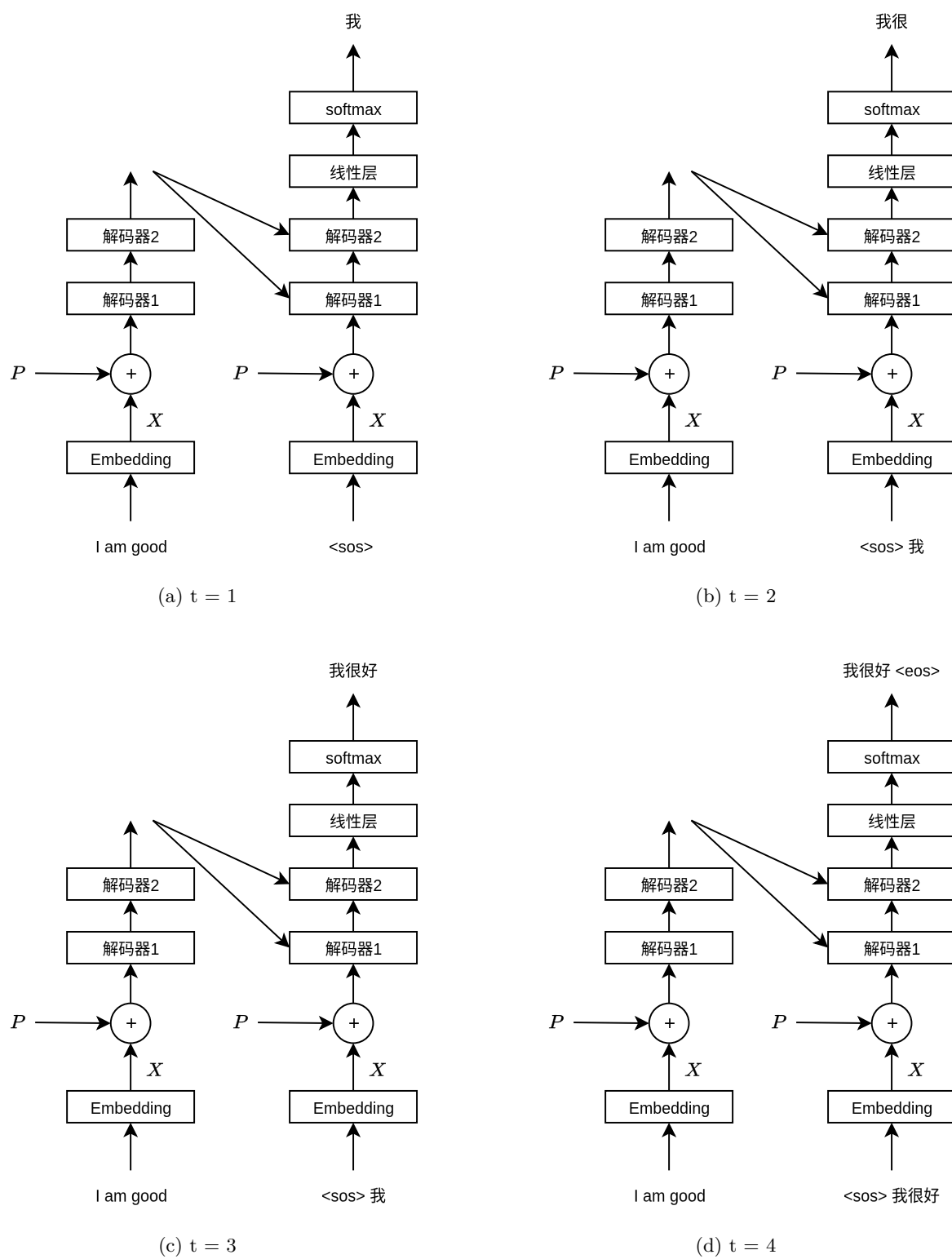


图 9.3: 解码器流程说明

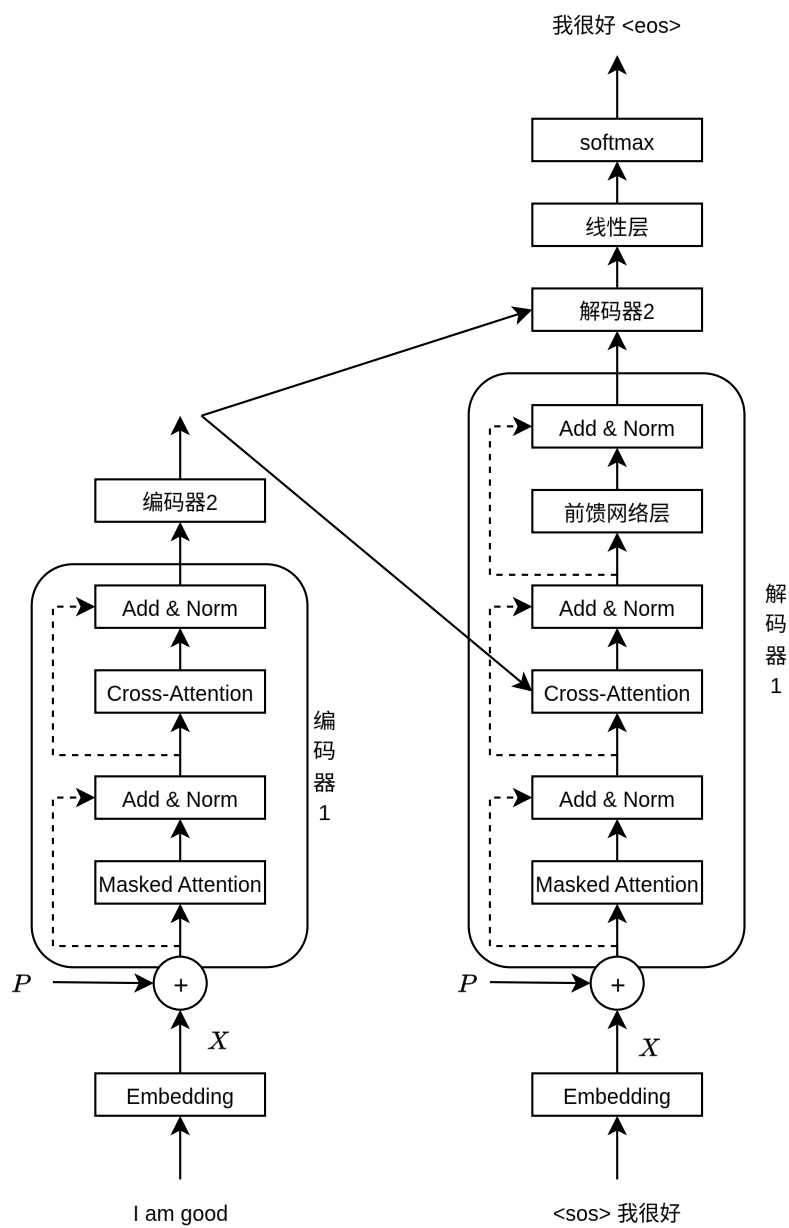


图 9.4: Transformer 总结构

第二部分

大模型

从 Transformer 到智能体

在前面的章节中，我们从感知机一路走到了 Transformer，理解了大型语言模型的技术基础。从本章开始，我们将进入一个全新的领域——如何“用好”大模型，让它真正为我们做事。在进入具体内容之前，我们先梳理一下这条技术演进的时间脉络。

2017：Transformer 诞生

Google 发表论文《Attention Is All You Need》，提出 Transformer 架构。这个架构抛弃了 RNN 的序列处理方式，完全基于注意力机制实现并行计算，为后来所有大语言模型奠定了基石。

2020：GPT-3 与少样本学习

OpenAI 发布 GPT-3，拥有 1750 亿参数。GPT-3 最令人惊讶的能力是——它不需要微调，仅凭提示中的几个示例（Few-shot）就能完成各种任务。这让人们意识到，大模型本身就是一个通用的任务求解器，关键在于你怎么“提问”。

2022：ChatGPT 与提示词工程

2022 年底，ChatGPT 横空出世，大语言模型从实验室走向大众。与此同时，提示词工程（Prompt Engineering）成为一门显学——人们发现，同一个问题换一种问法，模型的回答质量天差地别。Zero-shot、Few-shot、Chain-of-Thought 等技术相继被提出和普及。

2023：思维链与 RAG

思维链（CoT）技术进一步成熟，让模型能够处理复杂的推理任务。与此同时，检索增强生成（RAG）技术兴起，解决了大模型知识截止和幻觉问题——模型可以“翻书”回答问题，而不是仅凭记忆。这一年，工具使用（Tool Use）的概念也被引入，模型开始能够调用外部 API。

2024–2025：智能体时代

Agent（智能体）成为最热门的方向。模型不再只是回答问题，而是能够自主规划、使用工具、执行代码、操作文件，完成复杂的实际任务。Claude Codex、OpenAI Codex 等编码智能体相继出现，AI 从”对话助手”进化为”行动执行者”。

技术演进的本质

回顾这条时间线，我们可以看到一条清晰的演进逻辑：

阶段	核心问题	解决方案
提示词工程	如何问对问题	Zero-shot / Few-shot / CoT
RAG	如何获取最新知识	检索 + 生成
工具使用	如何与外部交互	Function Calling
智能体	如何自主完成任务	规划 + 工具 + 记忆

本质上，这是一个让大模型从”能说”到”能做”的过程。提示词工程解决了”怎么问”的问题，RAG 解决了”知道什么”的问题，工具使用解决了”能做什么”的问题，而智能体将这些能力整合在一起，让 AI 真正成为一个能够独立完成任务的助手。

接下来，我们将逐一深入这些技术，理解它们的原理和实现。

第十章 提示词工程

10.1 什么是提示词工程

在前面的章节中，我们学习了从感知机到 Transformer 的深度学习技术。当我们训练好一个大型语言模型（如 GPT、Claude 等）后，如何使用它就成为了一个关键问题。提示词工程（Prompt Engineering）就是研究如何设计和优化输入提示（Prompt），以引导模型生成更准确、更有用的输出。

打个比方，如果把大语言模型比作一个知识渊博的专家，那么提示词就是你向这位专家提问的方式。同一个问题，不同的问法可能得到完全不同质量的答案。提示词工程就是学习如何“问对问题”的艺术。

10.2 零样本提示

零样本提示（Zero-shot Prompting）是最简单的提示方式，就是直接告诉模型你要什么，不提供任何示例。

提示：将以下英文翻译成中文：

"Hello, how are you?"

输出：你好，最近怎么样？

这种方式就像你直接问一个翻译：“这句话什么意思？”模型会基于它预训练时学到的知识直接给出答案。

10.3 少样本提示

少样本提示（Few-shot Prompting）是在提示中提供几个示例，让模型学习你想要的格式和模式。这就像教新员工时，你会先做几个示范，然后让他们模仿。

提示：

英文：I love this movie -> 情感：正面

英文：This is terrible -> 情感：负面

英文: It's okay -> 情感: 中性

英文: What an amazing experience -> 情感:

输出: 正面

在这个例子中, 我们提供了三个示例, 模型就能理解我们想要进行情感分类任务, 并按照相同的格式输出。

少样本提示之所以有效, 是因为大型语言模型具有强大的上下文学习能力 (In-context Learning)。模型虽然没有在训练时专门学习这个任务, 但它能够根据提示中的示例, 推断出任务的模式和期望的输出格式。

10.4 思维链提示

思维链提示 (Chain-of-Thought Prompting, 简称 CoT) 是一种让模型”逐步思考”的技术。对于复杂的推理任务, 直接要求答案往往效果不好, 但如果让模型展示思考过程, 准确率会大幅提升。

这就像做数学题时, 老师会要求你写出解题步骤, 而不仅仅是答案。写出步骤不仅能帮助你理清思路, 也更容易发现错误。

提示:

问题: 小明有 5 个苹果, 他给了小红 2 个, 又买了 3 个, 现在他有几个苹果?

让我们一步一步思考。

输出:

1. 小明最初有 5 个苹果
2. 他给了小红 2 个, 剩下 $5 - 2 = 3$ 个
3. 他又买了 3 个, 所以 $3 + 3 = 6$ 个
4. 答案是: 小明现在有 6 个苹果

关键在于”让我们一步一步思考”这句话, 它引导模型展示推理过程, 而不是直接猜测答案。

10.5 提示词工程的实践技巧

10.5.1 明确具体

模糊的提示往往得到模糊的回答。好的提示应该明确说明你的需求:

差的提示: 写点关于Python的内容

好的提示: 用Python写一个函数, 输入是一个整数列表, 输出是列表中的最大值。要求使用for循环实现, 并添加类型注解和文档字符串。

10.5.2 指定格式

如果你需要特定格式的输出，明确告诉模型：

提示：列出三个Python的优点，用JSON格式输出，
每个优点包含"title"和"description"字段。

10.5.3 角色扮演

让模型扮演特定角色，可以得到更专业的回答：

提示：你是一位有10年经验的高级Python开发工程师。
请review以下代码，指出潜在的问题和改进建议：
[代码内容]

10.5.4 分解复杂任务

对于复杂的任务，将其分解为多个简单的步骤：

提示：

任务：分析一篇技术文章并生成摘要

步骤1：提取文章的核心观点

步骤2：为每个观点写一句话总结

步骤3：将三个总结组合成一段100字以内的摘要

文章内容：[文章内容]

10.6 提示词工程的本质

提示词工程之所以重要，是因为大型语言模型本质上是一个条件概率模型。给定输入序列 $x_1, x_2, \dots, x_n$ ，模型预测下一个词 x_{n+1} 的概率分布：

$$P(x_{n+1} | x_1, x_2, \dots, x_n)$$

提示词就是这个条件序列。一个好的提示能够更好地激活模型中相关的知识，引导模型生成更符合期望的输出。这就像在图书馆找书，如果你能给图书管理员更精确的描述，他就更容易找到你想要的书。

提示词工程不需要修改模型的参数，只需要调整输入，因此它是一种非常灵活和低成本的方式来优化模型的表现。

第十一章 检索增强生成 (RAG)

11.1 大语言模型的局限性

在前面的章节中，我们学习了大型语言模型（LLM）的强大能力。然而，LLM 也存在一些固有的局限性：

- **知识截止**：模型的训练数据有截止日期，无法获取最新信息
- **幻觉问题**：模型可能会生成看似合理但实际上错误的信息
- **领域知识**：对于特定领域的私有数据，模型无法访问
- **上下文长度**：模型的上下文窗口有限，无法处理超长文本

打个比方，LLM 就像一个博学的人，但他的知识停留在几年前，而且无法查阅最新的资料。如果你问他今天发生了什么，他只能凭记忆回答，可能会出错。

11.2 什么是 RAG

检索增强生成 (Retrieval-Augmented Generation, 简称 RAG) 是一种结合检索和生成的技术。它的核心思想是：在生成回答之前，先从外部知识库中检索相关信息，然后将检索到的信息作为上下文提供给 LLM，让模型基于这些真实的信息生成回答。

这就像考试时允许翻书。你不需要记住所有知识，只需要知道如何查找相关信息，然后基于这些信息回答问题。

RAG 的基本流程如下：

1. 用户提出问题
2. 系统从知识库中检索相关文档
3. 将问题和检索到的文档一起提供给 LLM
4. LLM 基于检索到的信息生成回答

11.3 文本嵌入

RAG 的第一步是将文本转换为向量表示，这个过程称为文本嵌入 (Text Embedding)。

想象你在图书馆整理书籍。你会把相似主题的书放在同一个书架上。文本嵌入就是给每本书 (文本) 一个坐标 (向量)，使得内容相似的文本在向量空间中距离较近。

数学上，嵌入模型 f 将文本 t 映射到一个 d 维向量：

$$f : t \rightarrow \mathbb{R}^d$$

例如：

文本1: "机器学习是人工智能的一个分支"

文本2: "深度学习属于机器学习领域"

文本3: "今天天气很好"

嵌入后：

向量1: [0.2, 0.8, -0.3, ...]

向量2: [0.3, 0.7, -0.2, ...] // 与向量1相似

向量3: [-0.5, 0.1, 0.9, ...] // 与前两个不相似

常用的相似度度量方法是余弦相似度：

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

余弦值越接近 1，表示两个向量越相似。

11.4 向量数据库

检索到的文档需要存储在某个地方，这就是向量数据库的作用。向量数据库专门用于存储和检索高维向量。

打个比方，传统数据库就像电话簿，你通过名字 (精确匹配) 查找电话号码。向量数据库更像是一个智能地图，你说“我想找个附近好吃的中餐馆”，它会返回地理位置相近的选项。

常见的向量数据库包括：

- FAISS (Facebook AI Similarity Search)
- Pinecone
- Milvus
- ChromaDB

向量数据库的核心功能是近似最近邻搜索 (Approximate Nearest Neighbor, ANN)。当查询向量到来时，数据库快速找到与之最相似的 k 个向量。

11.5 RAG 的工作流程

让我们通过一个完整的例子来理解 RAG 的工作流程。

假设你有一个公司内部的技术文档库，员工可以提问技术问题。

11.5.1 索引阶段

首先，需要对文档进行预处理和索引：

文档1: Python 的 GIL (全局解释器锁) 确保同一时刻只有一个线程执行 Python 字节码。

文档2: 使用 `multiprocessing` 模块可以绕过 GIL 的限制，实现真正的并行计算。

文档3: `asyncio` 是 Python 的异步编程框架，适合 I/O 密集型任务。

处理步骤：

1. **分块**：将长文档切分成较小的文本块（如 500 字一块）
2. **嵌入**：使用嵌入模型（如 OpenAI 的 `text-embedding-ada-002`）将每个文本块转换为向量
3. **存储**：将向量和对应的文本存储到向量数据库

11.5.2 查询阶段

当用户提问时：

用户问题: "如何在 Python 中实现并行计算？"

处理步骤：

1. **嵌入问题**：将用户问题转换为向量
2. **检索**：在向量数据库中搜索最相似的文本块（如 top-3）
3. **构建提示**：将问题和检索到的文本块组合成提示
4. **生成回答**：LLM 基于提供的上下文生成回答

构建的提示可能如下：

基于以下信息回答问题：

[检索到的文档块1]

使用 `multiprocessing` 模块可以绕过 GIL 的限制，

实现真正的并行计算。

[检索到的文档块2]

`asyncio` 是 Python 的异步编程框架，适合 I/O 密集型任务。

问题：如何在 Python 中实现并行计算？

回答：

LLM 会基于检索到的信息生成：

在 Python 中实现并行计算，可以使用 `multiprocessing` 模块。这个模块可以绕过 GIL（全局解释器锁）的限制，实现真正的并行计算。如果需要处理 I/O 密集型任务，可以考虑使用 `asyncio` 异步编程框架。

11.6 RAG 的优势

RAG 技术具有以下优势：

- **知识实时更新**：只需更新知识库，无需重新训练模型
- **减少幻觉**：模型基于真实文档回答，可以追溯信息来源
- **领域特定**：可以构建特定领域的知识库，提供专业回答
- **成本效益**：无需微调模型，只需维护知识库

11.7 RAG 的挑战

尽管 RAG 很强大，但也面临一些挑战：

- **检索质量**：如果检索到的文档不相关，会影响生成质量
- **分块策略**：如何合理切分文档是个技术活
- **上下文长度**：检索太多文档会超出模型的上下文窗口
- **延迟**：检索和生成两个步骤增加了响应时间

针对这些问题，研究者提出了许多改进方法，如：

- **混合检索**：结合关键词检索和向量检索
- **重排序**：对检索结果进行二次排序
- **查询改写**：优化用户查询以提高检索效果

第十二章 智能体与工具使用

12.1 从对话到行动

在前面的章节中，我们学习了提示词工程和 RAG 技术。这些技术让 LLM 能够更好地理解和回答问题。但是，LLM 本身只能生成文本，无法直接与外部世界交互。

想象你是一个被关在房间里的人，你知识渊博，但你无法打电话、无法上网、无法操作电脑。你只能通过门缝递纸条与人交流。这就是 LLM 的处境——它有强大的推理能力，但缺乏执行能力。

智能体（Agent）技术就是给 LLM 装上”手和脚”，让它不仅能思考，还能行动。

12.2 什么是智能体

智能体（Agent）是一个能够感知环境并采取行动以实现目标的系统。在 AI 领域，LLM 智能体通常包含以下组件：

- **大脑**：LLM 作为核心推理引擎
- **工具**：可以调用的外部功能（如搜索引擎、代码执行器、API 等）
- **记忆**：短期记忆（对话历史）和长期记忆（文件系统）
- **规划**：将复杂任务分解为可执行的步骤

打个比方，LLM 是一个聪明的大脑，智能体就是给这个大脑配上工具箱、记事本和计划表，让它能够完成实际任务。

12.3 Agent 循环：智能体的心跳

12.3.1 ReAct 框架

ReAct（Reasoning and Acting）是最经典的智能体框架，它将推理和行动结合起来。ReAct 的核心思想是让 LLM 交替进行”思考”（Reasoning）和”行动”（Acting），形成一个 Thought-Action-Observation 的循环：

用户：帮我查一下特斯拉当前的股价，并分析最近走势

Thought 1: 我需要先获取特斯拉的当前股价信息

Action 1: 调用股票查询工具，查询 TSLA

Observation 1: 特斯拉当前股价 245.50 美元

Thought 2: 现在我需要获取最近的历史数据来分析走势

Action 2: 调用历史数据工具，查询 TSLA 近30天数据

Observation 2: [返回30天的股价数据]

Thought 3: 根据数据，我可以分析走势了

Final Answer: 特斯拉当前股价 245.50 美元，
最近30天呈现上涨趋势，涨幅约 11.6%...

12.3.2 真实的 Agent 循环

上面的 ReAct 示例做了简化。在实际的智能体系统中（如 Claude Code），Agent 循环要复杂得多。下面是一个真实 Agent 循环的核心流程：

1. **构建系统提示词**：组装系统提示词，包含工具定义、环境信息、记忆等
2. **调用模型**：将对话历史发送给 LLM，获取模型响应
3. **解析工具调用**：检查模型输出中是否包含工具调用请求
4. **权限检查**：在执行工具前，验证用户是否授权
5. **执行工具**：调用相应的工具函数，获取结果
6. **注入结果**：将工具执行结果作为新消息加入对话历史
7. **判断终止**：如果没有工具调用，说明模型认为任务完成，退出循环；否则回到第 2 步

这个过程可以用下面的伪代码表示：

```
function agentLoop(messages, tools, systemPrompt):  
    while true:  
        // 1. 调用 LLM  
        response = callModel(messages, tools, systemPrompt)  
  
        // 2. 检查是否有工具调用  
        if response has no tool_use blocks:  
            return response // 任务完成，退出循环
```



```
// 3. 执行所有工具调用
for each tool_use in response:
    tool = findTool(tool_use.name)
    result = tool.execute(tool_use.input)
    messages.append(tool_result)

// 4. 继续循环，让模型看到工具结果
```

这个循环看起来简单，但实际实现中有许多精妙的设计。

12.3.3 流式工具执行

在 Claude Code 中，工具执行并不是等模型完全输出后才开始的。当模型的输出以流 (stream) 的形式返回时，一旦某个工具调用的 JSON 参数完整接收，系统就会立即开始执行该工具，而不需要等待模型输出完毕。

打个比方，这就像餐厅里服务员还没记完你的所有菜，就已经把先点好的菜传给厨房了。这种**流式工具执行** (Streaming Tool Execution) 大大减少了等待时间。

此外，标记为“并发安全”的工具可以并行执行。比如同时搜索多个文件，不需要等一个搜索完了再搜索下一个。

12.3.4 错误恢复

真实的 Agent 循环需要处理各种异常情况。Claude Code 中实现了多层恢复机制：

- **输出截断**：当模型输出达到 token 上限时，自动提高限制并重试（最多 3 次）
- **上下文过长**：当提示词超过模型上下文窗口时，自动压缩对话历史后重试
- **API 错误**：网络或服务端错误时，自动切换到备用模型重试

这些恢复机制对用户是透明的，确保智能体能够稳定运行。

12.3.5 步数限制与“末日循环”检测

一个容易被忽视的问题是：如果智能体陷入死循环怎么办？比如反复调用同一个工具、反复修改同一个文件，却永远无法完成任务。

OpenCode 的解决方案是设置**步数上限**（默认 25 步）。当智能体连续执行超过这个上限时，系统会强制终止循环，并向用户报告当前进度。这就像给智能体设了一个“最大工作时间”，超时就必须停下来汇报。

Gemini CLI 则实现了“**末日循环**”检测 (Doom Loop Detection)：系统会监控智能体的行为模式，如果发现它连续多次执行相同的操作（比如连续 3 次调用同一个工具、传入相同的参数），就会主动介入并提醒用户。

12.3.6 持久化输入与会话恢复

在 OpenCode 的 V2 架构中，引入了一个精巧的设计——**持久化输入** (Durable Input)。用户发送的消息不会直接交给模型，而是先存入一个数据库“收件箱”，然后在安全的边界点被“提升”到对话中。

这就像邮件系统：你发出的邮件先进入对方的收件箱，对方在方便的时候才打开阅读。这种设计的好处是，即使进程崩溃，未处理的消息也不会丢失，重启后可以继续处理。

12.4 工具系统

工具使用 (Tool Use) 是智能体的核心能力。它允许 LLM 在对话过程中调用外部工具来获取信息或执行操作。

12.4.1 工具的定义

每个工具都需要告诉 LLM 三件事：我叫什么、我能做什么、你需要给我什么参数。这些信息通过 JSON Schema 来定义：

```
{
  "name": "get_weather",
  "description": "获取指定城市的天气信息",
  "input_schema": {
    "type": "object",
    "properties": {
      "city": {
        "type": "string",
        "description": "城市名称"
      },
      "date": {
        "type": "string",
        "description": "日期，如 'today', 'tomorrow'"
      }
    },
    "required": ["city"]
  }
}
```

在 Claude Code 中，工具的定义更加丰富。一个编码智能体通常配备 60 多个工具，涵盖了文件操作、代码搜索、命令执行、网络访问等方方面面。以下是 Claude Code 中的部分核心工具：

工具名称	功能描述
Bash	执行终端命令
Read	读取文件内容
Write	创建或覆盖文件
Edit	精确替换文件中的文本
Glob	按模式搜索文件名
Grep	按内容搜索文件
Agent	启动子智能体
WebFetch	获取网页内容
WebSearch	搜索引擎搜索
LSP	代码智能（跳转定义等）

表 12.1: Claude Code 核心工具列表

12.4.2 工具调用的完整生命周期

一个工具调用从开始到结束，要经过以下步骤：

- 1. **模型决策**：LLM 在输出中生成一个 `tool_use` 块，包含工具名和参数
- 2. **工具查找**：系统根据名称在已注册的工具列表中查找对应工具
- 3. **输入验证**：用 JSON Schema 验证模型传入的参数是否合法
- 4. **权限检查**：检查用户是否授权该操作（通过 Hook 系统或用户确认）
- 5. **执行工具**：调用工具的 `execute` 方法，获取结果
- 6. **结果格式化**：将结果转换为 `tool_result` 格式
- 7. **注入对话**：将结果作为用户消息加入对话历史，供模型下次读取

用一个具体的例子来说明。当用户说”帮我看看 `main.py` 的内容”时：

```
// 第1步：模型输出 tool_use 块
{
  "type": "tool_use",
  "id": "toolu_01ABC",
  "name": "Read",
  "input": { "file_path": "/home/user/main.py" }
}

// 第2-5步：系统查找 Read 工具，验证参数，执行读取
```

```
// 读取结果:
"def main():
    print('Hello, World!')

if __name__ == '__main__':
    main()"

// 第6-7步: 格式化并注入对话历史
{
  "type": "tool_result",
  "tool_use_id": "toolu_01ABC",
  "content": "def main():\n    print('Hello, World!')\n..."
}
```

模型看到这个结果后, 就可以基于文件内容回答用户的问题了。

12.4.3 工具执行策略

不同的智能体系统在工具执行策略上有各自的设计。

并发控制 在 Claude Code 中, 标记为“并发安全”的工具可以并行执行。而 Gemini CLI 采用了一种更灵活的方式——它在每个工具的参数中自动添加一个 `wait_for_previous` 参数, 让模型自己决定哪些工具可以并行、哪些必须串行。

```
// Gemini CLI 自动为工具添加的参数
{
  "name": "read_file",
  "parameters": {
    "path": {"type": "string"},
    "wait_for_previous": {
      "type": "boolean",
      "description": "是否等待前一个工具执行完毕"
    }
  }
}
```

这种设计的巧妙之处在于: 把并发控制的决策权交给了模型。模型可以根据任务依赖关系, 智能地决定执行顺序。

后台执行 Gemini CLI 的 Shell 工具支持后台执行。当命令耗时较长时（比如编译大型项目），智能体可以让命令在后台运行，同时继续处理其他任务。系统通过 PID 追踪来管理后台进程，确保能够获取最终结果。

```
// 后台执行示例
{
  "command": "make build",
  "is_background": true
}
// 系统返回：进程已在后台启动，PID: 12345
```

工具输出管理 工具的输出可能非常长（比如读取一个大文件）。OpenCode 实现了**有界输出机制**：工具结果被分为“预览”和“完整内容”两部分。预览保留在对话历史中供模型参考，完整内容则写入临时文件。如果模型需要完整内容，可以通过专门的工具读取。

这就像看报告时，摘要直接附在邮件正文里，完整报告作为附件。收件人先看摘要，需要时再打开附件。

12.4.4 权限与安全

智能体拥有执行能力后，安全性就变得至关重要。不同的智能体系统实现了不同层次的权限控制。

审批模式 Gemini CLI 定义了四种审批模式，从严格到宽松：

- **DEFAULT**：每次工具调用都需要用户确认
- **AUTO_EDIT**：自动批准文件编辑操作，其他操作仍需确认
- **PLAN**：计划模式，智能体只能读取和分析，不能修改文件
- **YOLO**：自动批准所有操作（适合完全信任的场景）

这种分级设计让用户可以根据任务的危险程度选择合适的模式。比如日常编码可以用 AUTO_EDIT，而处理敏感数据时用 DEFAULT。

IAM 风格的权限规则 OpenCode 采用了类似云服务的 IAM (Identity and Access Management) 权限模型。每个智能体都有自己的权限规则集：

```
// OpenCode 的权限规则示例
{
  "effect": "deny",
  "action": "bash",
```

```
"resource": "rm -rf *"
}
```

这种规则可以在智能体启动时就过滤掉不允许的工具，而不是等到执行时才发现权限不足。这就像门禁系统：不给你某间房的门禁卡，你就根本进不去，而不是进去了才发现门被锁了。

平台特定的沙箱 Gemini CLI 实现了平台特定的沙箱系统，针对 Linux、macOS、Windows 分别实现了不同的安全隔离机制。这包括文件系统访问限制、网络访问控制、进程隔离等。

主动权限建议 一个有趣的创新是 Gemini CLI 的**主动权限建议**功能。当智能体检测到某个操作可能需要特定权限时，它会主动建议用户授权，而不是等到操作失败后才提示。这提升了用户体验，减少了中断。

12.5 函数调用：连接模型与工具的桥梁

12.5.1 函数调用的工作原理

函数调用（Function Calling）是工具使用的底层实现机制。我们在第 9 章学习了 Transformer 的解码过程——模型逐词生成文本。函数调用在此基础上做了扩展：模型的输出不再仅仅是文本，还可以是结构化的工具调用指令。

具体来说，当我们将工具定义发送给 API 时，模型会在词表中增加一些特殊的 token，如 `tool_use` 标记。当模型决定调用工具时，它会生成这些特殊 token，而不是普通的文字。

```
// 发送给 API 的请求
{
  "model": "claude-sonnet-4-20250514",
  "messages": [{"role": "user", "content": "计算 123 * 456"}],
  "tools": [{
    "name": "calculate",
    "description": "执行数学计算",
    "input_schema": {
      "type": "object",
      "properties": {
        "expression": {"type": "string"}
      },
      "required": ["expression"]
    }
  ]
}
```

```
// API 返回的响应
{
  "content": [
    {"type": "text", "text": "让我帮你计算一下。"},
    {"type": "tool_use", "id": "toolu_01ABC",
     "name": "calculate",
     "input": {"expression": "123 * 456"}}
  ]
}
```

12.5.2 多轮工具调用

一个强大的智能体往往需要连续调用多个工具。每次工具调用的结果会被加入对话历史，模型基于更新后的历史继续推理，形成多轮交互：

用户：帮我创建一个 Python 项目

第1轮 - 模型调用 Bash 工具：mkdir my_project

第1轮 - 工具返回：创建成功

第2轮 - 模型调用 Write 工具：写入 main.py

第2轮 - 工具返回：写入成功

第3轮 - 模型调用 Write 工具：写入 requirements.txt

第3轮 - 工具返回：写入成功

第4轮 - 模型调用 Bash 工具：python main.py 验证

第4轮 - 工具返回：Hello, World!

第5轮 - 模型输出文本：项目已创建完成，包含以下文件...
(没有工具调用，循环结束)

12.6 系统提示词：智能体的灵魂

12.6.1 系统提示词的作用

如果说工具是智能体的”手和脚”，那么系统提示词（System Prompt）就是智能体的”灵魂”。它定义了智能体的身份、行为准则和工作方式。

系统提示词与普通提示词的区别在于：它在每次对话开始时就设定好了，贯穿整个对话过程，模型会始终遵循其中的指示。

12.6.2 系统提示词的构建

不同的智能体系统在系统提示词的构建上采用了不同的策略。

Claude Code：模块化组装 在 Claude Code 中，系统提示词由多个”片段”动态组装而成：

1. **介绍部分**：定义模型的身份
2. **行为准则**：规定模型应该如何行事
3. **工具说明**：描述每个工具的使用方法
4. **环境信息**：当前日期、工作目录、操作系统等
5. **项目上下文**：CLAUDE.md 中的项目约定
6. **记忆内容**：智能体记住的用户偏好和项目知识

Gemini CLI：分层记忆系统 Gemini CLI 实现了**分层记忆**（Hierarchical Memory）：

- **全局记忆**：7.gemini/GEMINI.md，适用于所有项目
- **扩展记忆**：来自 VSCode 等 IDE 的记忆
- **项目记忆**：.gemini/GEMINI.md，仅适用于当前项目

这种分层设计让智能体能够区分通用知识和项目特定知识。

OpenCode：Provider 特定的提示词 OpenCode 支持多个 LLM 提供商（Anthropic、OpenAI、Gemini 等），不同模型的”性格”和能力不同，因此 OpenCode 为每个 Provider 准备了不同的系统提示词模板：

```
function provider(model) {  
  if (model.includes("gpt-4")) return PROMPT_GPT4  
  if (model.includes("gemini")) return PROMPT_GEMINI  
  if (model.includes("claude")) return PROMPT_CLAUDE  
  return PROMPT_DEFAULT  
}
```

这种设计的理由是：不同模型对指令的理解方式不同，针对性的提示词能更好地发挥每个模型的优势。

上下文代数 OpenCode 的 V2 架构引入了**系统上下文代数** (System Context Algebra)。系统提示词不再是一个字符串，而是由多个”上下文源” (Context Source) 组合而成的代数结构。每个上下文源可以独立更新，系统会自动计算最终的提示词。

这就像化学反应：不同的”元素” (上下文源) 按照一定的”化学方程式” (组合规则) 反应生成最终的”化合物” (系统提示词)。当某个元素变化时，只需要重新计算，而不需要从头开始。

12.7 子智能体：分工协作

12.7.1 为什么需要子智能体

复杂的任务往往超出单个智能体的能力范围。就像一个公司需要不同部门协作一样，智能体也需要”分身”来并行处理不同的子任务。

例如，当用户说”帮我重构认证模块并更新测试”时，主智能体可以：

- 启动一个子智能体去分析认证模块的代码结构
- 同时启动另一个子智能体去查找相关的测试文件
- 收集两个子智能体的结果后，再制定重构方案

12.7.2 子智能体的隔离

子智能体在独立的上下文中运行，这就像一个员工被派到一间独立的办公室工作——他有自己的工作台 (消息历史)，看不到其他员工的工作，但他继承了公司的规章制度 (系统提示词)。

在 Claude Code 中，子智能体的上下文隔离通过以下机制实现：

- **独立的消息历史**：子智能体有自己的对话记录，不会污染主对话
- **独立的文件缓存**：子智能体读取文件时建立自己的缓存
- **关联的中止控制**：主智能体中止时，子智能体也会自动中止
- **独立的 Agent ID**：每个子智能体有唯一标识，便于追踪

12.7.3 多智能体协调模式

在更复杂的场景下，Claude Code 支持”协调者-工人” (Coordinator-Worker) 模式：

- **协调者 (Coordinator)**：负责理解任务、分配工作、综合结果。它不直接执行具体操作，而是通过派遣工人来完成任务
- **工人 (Worker)**：接收具体任务，拥有完整的工具集，独立执行任务后将结果报告给协调者

这就像一个项目经理和开发团队的关系：项目经理负责需求分析和任务分配，开发人员负责具体实现，完成后向项目经理汇报。

12.7.4 Agent 间通信协议

当多个智能体需要协作时，它们之间如何通信是一个关键问题。Google 提出了 **A2A** (Agent-to-Agent) 协议，用于规范智能体之间的通信。

在 Gemini CLI 中，子智能体有一个重要的限制：**不允许递归调用**。也就是说，子智能体不能再启动子智能体。这是为了防止无限嵌套导致的资源失控。

主智能体

子智能体 A (分析代码结构)

不能再启动子智能体

子智能体 B (查找测试文件)

不能再启动子智能体

汇总结果

这种设计虽然简单，但有效地避免了”俄罗斯套娃”式的无限嵌套问题。

12.8 上下文管理

上下文管理是智能体系统中一个容易被低估的难题。它不仅关乎对话历史的存储，更影响智能体的推理质量和成本。

12.8.1 上下文压缩

当对话历史接近模型的上下文窗口上限时，所有智能体系统都会进行压缩。但压缩的策略各有不同。

Claude Code 的压缩相对直接：将早期消息总结为摘要，保留最近的对话。

OpenCode 的压缩更加结构化，它会生成一个包含以下字段的摘要：

- **目标**：用户想要完成什么
- **约束**：有哪些限制条件
- **进展**：已完成、进行中、被阻塞的任务
- **关键决策**：做出了哪些重要决定
- **下一步**：接下来应该做什么
- **相关文件**：涉及哪些文件

这种结构化的压缩能更好地保留关键信息，避免智能体在压缩后”忘记”重要上下文。

12.8.2 历史加固

Gemini CLI 实现了一个独特的功能——**历史加固** (History Hardening)。在将对话历史发送给模型之前，系统会对历史进行“清洗”，移除或修正可能误导模型的内容。

打个比方，这就像在把会议纪要发给参会者之前，先由助理检查一遍，修正其中的错别字和歧义表述，确保每个人看到的都是清晰准确的信息。

12.8.3 上下文纪元

OpenCode V2 引入了**上下文纪元** (Context Epoch) 的概念。一个纪元是指系统提示词保持不变的一段时间跨度。当发生以下事件时，会开启新的纪元：

- 上下文压缩
- 切换模型或提供商
- 切换智能体

每个纪元都有一个“基线” (Baseline)，记录了该纪元开始时的系统上下文状态。这就像版本控制系统中的“快照”——你可以随时回到某个快照的状态。

这种设计的好处是：在一个纪元内，系统提示词是不可变的，这保证了模型推理的一致性。当需要变更时，通过纪元切换来管理，而不是在对话中途偷偷修改提示词。

12.8.4 图结构上下文

Gemini CLI 采用了最复杂的上下文管理方案——**图结构上下文** (Graph-based Context)。传统的上下文管理是一个线性的消息列表，而 Gemini CLI 将上下文表示为一个有向图：

- **节点** (Node)：每条消息是一个节点
- **处理器** (Processor)：对节点进行变换的管道
- **工作缓冲区** (Working Buffer)：处理器可以修改的可变状态

这种设计允许对上下文进行复杂的变换操作，比如：重新排序消息、注入额外的上下文、过滤不相关的内容等。

打个比方，线性上下文就像一条流水线，原料进去，产品出来。而图结构上下文更像一个化工厂，原料可以在不同的反应罐之间流转，经过各种处理，最终生成产品。

12.9 记忆系统

12.9.1 短期记忆：对话历史

短期记忆就是当前对话的消息列表。它让智能体能够理解对话的连贯性。

用户：帮我查一下北京的天气

智能体：[调用天气工具] 北京今天晴，22°C

用户：那上海呢？ // 智能体从上下文知道还是在问天气

智能体：[调用天气工具] 上海今天多云，25°C

短期记忆面临的一个挑战是**上下文窗口限制**。模型的输入长度是有上限的，当对话很长时，早期的内容会被“遗忘”。为了解决这个问题，Claude Code 实现了**自动压缩**机制：当对话历史接近上下文窗口上限时，系统会自动将早期的消息压缩成摘要，保留关键信息，释放空间给新的内容。

12.9.2 长期记忆：文件系统

长期记忆让智能体能够跨会话记住信息。在 Claude Code 中，长期记忆通过文件系统实现。

具体来说，智能体会在项目目录下维护一个 `memory/` 文件夹，其中 `MEMORY.md` 是记忆的索引文件。智能体在工作过程中学到的重要信息（如用户偏好、项目约定、常见问题解决方案）会被写入这些文件。

下次启动时，智能体会自动读取这些记忆文件，恢复之前的“记忆”。

```
// 记忆文件示例 (~/.claude/projects/xxx/memory/MEMORY.md)
```

```
# 项目记忆
```

```
## 用户偏好
```

- 使用 TypeScript 而非 JavaScript
- 偏好函数式编程风格
- 测试框架使用 vitest

```
## 项目约定
```

- API 路由放在 `src/routes/` 目录
- 数据库操作使用 Prisma ORM
- 提交信息遵循 Conventional Commits

这种设计非常巧妙：它不需要修改模型参数，仅仅通过文件系统就实现了持久化记忆。就像一个工程师把自己的工作笔记放在抽屉里，下次上班时翻开笔记本就能回忆起之前的工作。

12.10 编码智能体：Claude Code

12.10.1 编码智能体的特点

编码智能体（Coding Agent）是专门用于编程任务的智能体。相比通用智能体，编码智能体需要更强大的代码理解和操作能力。

Claude Code 是 Anthropic 推出的编码智能体。它基于 Claude 模型，配备了丰富的开发工具，能够理解整个代码库的上下文，自主完成代码编写、调试、重构等复杂任务。

12.10.2 一个完整的工作流程

让我们通过一个完整的例子来理解 Claude Code 的工作流程。假设用户说：“帮我修复登录页面的 bug，点击按钮没有反应”。

Claude Code 的执行过程如下：

[思考] 用户报告登录按钮点击无反应，我需要先找到相关代码

[行动] 调用 `Grep` 工具，搜索 "login" 相关文件

[观察] 找到 `src/pages/Login.tsx`

[思考] 需要查看登录页面的代码

[行动] 调用 `Read` 工具，读取 `Login.tsx`

[观察] 发现 `handleSubmit` 函数中有一个拼写错误

[思考] 需要确认这个函数的调用方式

[行动] 调用 `LSP` 工具，查找 `handleSubmit` 的引用

[观察] 确认是按钮的 `onClick` 事件处理函数

[思考] 问题已定位，现在修复这个拼写错误

[行动] 调用 `Edit` 工具，将 `"formData"` 改为 `"formdata"`

[观察] 修改成功

[思考] 需要验证修复是否正确

[行动] 调用 `Bash` 工具，运行测试

[观察] 所有测试通过

[最终回答] 问题已修复。原因是 `handleSubmit` 函数中

`"formData"` 被拼写为 `"fromData"`，导致获取不到表单数据。

已修改 `src/pages/Login.tsx` 第42行，测试全部通过。

在这个过程中，Claude Code 自主完成了“搜索-阅读-分析-修复-验证”的完整流程，总共调用了 5 次工具，经历了 5 轮思考-行动循环。

12.10.3 技能系统

Claude Code 还实现了技能 (Skills) 系统，可以看作是针对特定任务的“专家模式”。每个技能定义了一组专用的提示词和工具集，当触发某个技能时，智能体会切换到该技能的工作方式。

例如，一个代码审查技能可能包含：

- 专门的审查提示词（关注安全漏洞、性能问题等）

- 限制只使用只读工具（Read、Grep、Glob），不允许修改文件
- 输出格式要求（按严重程度分类列出问题）

技能可以通过斜杠命令（如 `/review`）手动触发，也可以由智能体根据上下文自动选择。

12.11 智能体的挑战与展望

12.11.1 当前挑战

尽管智能体技术发展迅速，但仍面临一些挑战：

- **可靠性**：智能体可能会做出错误的决策或调用错误的工具，特别是在复杂的多步任务中，一个错误可能导致后续步骤全部失败
- **安全性**：需要防止智能体执行危险操作，如删除重要文件、泄露敏感信息等
- **效率**：多步推理和工具调用会增加响应时间和 API 成本
- **可控性**：用户希望了解智能体在做什么，并能在必要时介入

12.11.2 未来展望

智能体技术正在快速发展，未来的趋势包括：

- **多智能体协作**：多个专业化的智能体协同工作，就像一个高效的开发团队
- **自主学习**：智能体能够从经验中学习，不断改进策略
- **更丰富的工具生态**：MCP（Model Context Protocol）等标准让工具的接入更加规范化和便捷
- **个性化**：智能体能够适应不同用户的偏好和工作方式，通过记忆系统越用越懂你

智能体代表了 AI 从“能说”到“能做”的重要跨越。从提示词工程解决“怎么问”，到 RAG 解决“知道什么”，到工具使用解决“能做什么”，再到智能体将这些能力整合在一起——我们正在见证 AI 从“对话助手”进化为“行动执行者”的历史性转变。